



成绩 _____

北京航空航天大学
B E I H A N G U N I V E R S I T Y

《基于 ORACLE/ADO 的建模/仿真实 验》实验报告

院/系名称： 自动化科学与电气工程学院

专业名称： 自动化

学生学号： SY2503208

学生姓名： 张铃蕊

指导教师： 苑海涛

实验日期： 2025 年 11 月 12 日

2025 年 11 月



目录

一、实验目的	4
1.1 实验要求	4
1.2 实验目的	4
二、实验原理	4
2.1 粒子群优化算法（PSO）	4
2.1.1 算法介绍	4
2.1.2 算法数学模型	5
2.2 测试函数	6
2.2.1 单峰函数	6
2.2.2 多峰函数	11
2.3 实际问题-背包问题	20
2.3.1 问题定义	20
2.3.2 动态规划算法	21
三、系统架构	22
3.1 系统架构	22
3.2 核心模块	22
3.2.1 数据库管理模块	22
3.2.3 测试函数模块	23
3.2.4 背包问题求解模块	24
3.3 数据库设计	24
3.3.1 测试函数结果表	24
3.3.2 背包问题结果表	24
3.4 用户界面设计	25



3.4.1 主界面布局	25
3.4.2 可视化组件	27
四、实验过程与结果	28
4.1 系统功能验证	28
4.1.1 数据库连接功能测试	28
4.2 测试函数优化结果	29
4.2.1 作图结果	29
4.2.1 测试函数结果	33
4.3 背包问题求解结果	33
五、实验结果分析	34
5.1 测试函数收敛曲线分析	34
5.2 测试函数结果分析	35
5.3 背包问题结果分析	36
六、收获与总结	37



基于粒子群算法的优化问题求解系统实验报告

一、实验目的

1.1 实验要求

1. 针对一个或多个标准的测试函数,进行求解,将求解的结果存入到数据库中,并把这些数据展示在前端的界面上。测试函数和前端界面自由设计,要有可视化图形

2. 针对一个实际的优化问题进行求解和优化,将求解的结果存入到数据库中,并把这些数据展示在前端的界面上

1.2 实验目的

本实验旨在通过学习与编程过程深入理解粒子群优化(PSO)算法的原理和实现方法,通过标准测试函数评估 PSO 算法在不同问题上的表现,并且将优化算法应用于 0-1 背包问题等实际优化问题,同时在技术方面,学习设计并实现完整的优化问题求解系统,掌握数据库设计与应用,实现优化结果的持久化存储并通过图表直观展示算法收敛过程和优化效果。

二、实验原理

2.1 粒子群优化算法 (PSO)

2.1.1 算法介绍

粒子群优化算法是群智能优化领域的经典代表,由 Kennedy 和 Eberhart 于 1995 年正式提出。其核心灵感源于对自然界中鸟群觅食行为的深度观察与抽象模拟——鸟群在寻找食物时,个体并非孤立行动,而是通过感知同伴的位置、飞行方向等信息持续调整自身状态,最终形成协同高效的群体搜索模式。

该算法将优化问题的潜在解抽象为“粒子”,所有粒子共同构成一个动态的“粒子群”并在解空间中飞行。粒子的飞行状态由位置和速度两个核心参数描述,而搜索过程的本质的是粒子通过信息共享与协作不断更新自身状态的过程:每个粒子既会参考自身历史搜索过程中找到的最优解(个体极值),也会借鉴整个粒子群当前发现的全局最优解(全局极值),通过两者的动态平衡调整飞行方向与速度,

逐步逼近问题的最优解。

与传统优化算法相比，PSO 无需依赖问题的梯度信息，具有原理简洁、参数设置少、收敛速度快且鲁棒性强的特点。其核心优势在于通过模拟群体智能的协作机制，实现了全局探索与局部开发的有效平衡，因此被广泛应用于函数优化、神经网络训练、工程设计、路径规划等多个领域。

2.1.2 算法数学模型

粒子群优化算法的搜索过程通过明确粒子的位置与速度更新规则实现，其核心数学模型基于对粒子运动状态的动态调控，具体可从粒子的核心状态量、最优信息参考及更新公式三方面展开：

1. 粒子的核心状态量

在 PSO 算法中，每个粒子的运动状态由位置（Position） 和速度（Velocity）两个核心参数描述：

- 位置（记为 $x[i]$ ）：表示粒子在解空间中的坐标，对应优化问题的一个候选解，其维度与问题的决策变量维度一致；
- 速度（记为 $v[i]$ ）：反映粒子移动的方向和快慢，决定了粒子下一时刻位置的变化趋势。

2. 最优信息的参考依据

粒子的运动方向并非随机，而是基于两类关键最优信息动态调整：

- 个体最优（pbest，记为 $pbest[i]$ ）：指单个粒子在历史搜索过程中找到的最优位置（即对应最优候选解），反映粒子自身的搜索经验；
- 全局最优（gbest）：指整个粒子群在当前搜索阶段中发现的最优位置，代表群体协作得到的最优信息。

3. 位置与速度的更新公式

粒子通过持续更新速度和位置实现对解空间的搜索。

粒子在 $(t+1)$ 时刻的速度由三部分共同决定，有速度更新公式为：

$$v[i](t+1) = w \cdot v[i](t) + c_1 \cdot r_1 \cdot (pbest[i] - x[i](t)) + c_2 \cdot r_2 \cdot (gbest - x[i](t))$$

其中，第一项 $w \cdot v[i](t)$ 为惯性项，由惯性权重 w 控制上一时刻速度对当前速度的影响，体现粒子保持原有运动趋势的特性；第二项 $c_1 \cdot r_1 \cdot (pbest[i] - x[i](t))$ 为个体

学习项，表示粒子向自身历史最优位置 $pbest[i]$ 靠近的趋势，由个体学习因子 c_1 调节该趋势的强度；第三项 $c_2 \cdot r_2 \cdot (gbest - x[i](t))$ 为社会学习项，反映粒子向群体最优位置 $gbest$ 聚集的倾向，由社会学习因子 (c_2) 控制该倾向的程度； r_1 和 r_2 是区间 $[0,1]$ 内的随机数，用于增加搜索过程的随机性，避免算法陷入局部最优。

粒子在 $t+1$ 时刻的位置由当前位置与更新后的速度叠加得到位置更新公式为：

$$x[i](t+1) = x[i](t) + v[i](t+1)$$

这一规则直观反映了“速度决定位移”的物理逻辑，使粒子基于新速度调整在解空间中的位置，逐步逼近最优解。

2.1.3 算法流程

PSO 的算法流程为：

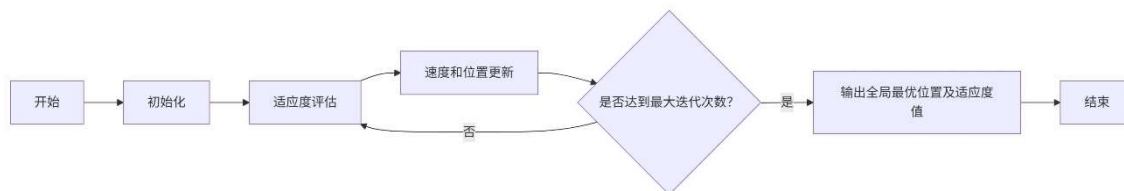


图 1 PSO 算法流程

如图 1 所示，PSO 算法流程中，初始化指随机生成粒子群的初始位置和速度并且设置算法参数(w , c_1 , c_2 , 最大迭代次数等)；适应度评估指计算每个粒子的适应度值、更新个体最优 $pbest$ 、更新全局最优 $gbest$ 。其余如图所示，最后会返回全局最优位置和对应的适应度值。

2.2 测试函数

测试函数是验证粒子群优化算法（PSO）寻优性能的核心工具，不同类型的测试函数能从不同维度考核算法的全局探索、局部开发及跳出局部最优等能力。本部分将测试函数分为单峰函数和多峰函数两类，其中单峰函数仅有一个全局最优解，主要用于测试算法的收敛精度和局部搜索能力；多峰函数存在多个局部最优解，重点考核算法的全局搜索与避免早熟收敛的能力。

2.2.1 单峰函数

单峰函数的函数图像呈“单峰”形态，无多余局部最优解，其优化过程可直观反映算法向最优解逼近的效率和最终精度，是验证算法基础寻优能力的首选。

1. Sphere 函数

Sphere函数（球函数） - 3D曲面图

★ 全局最优

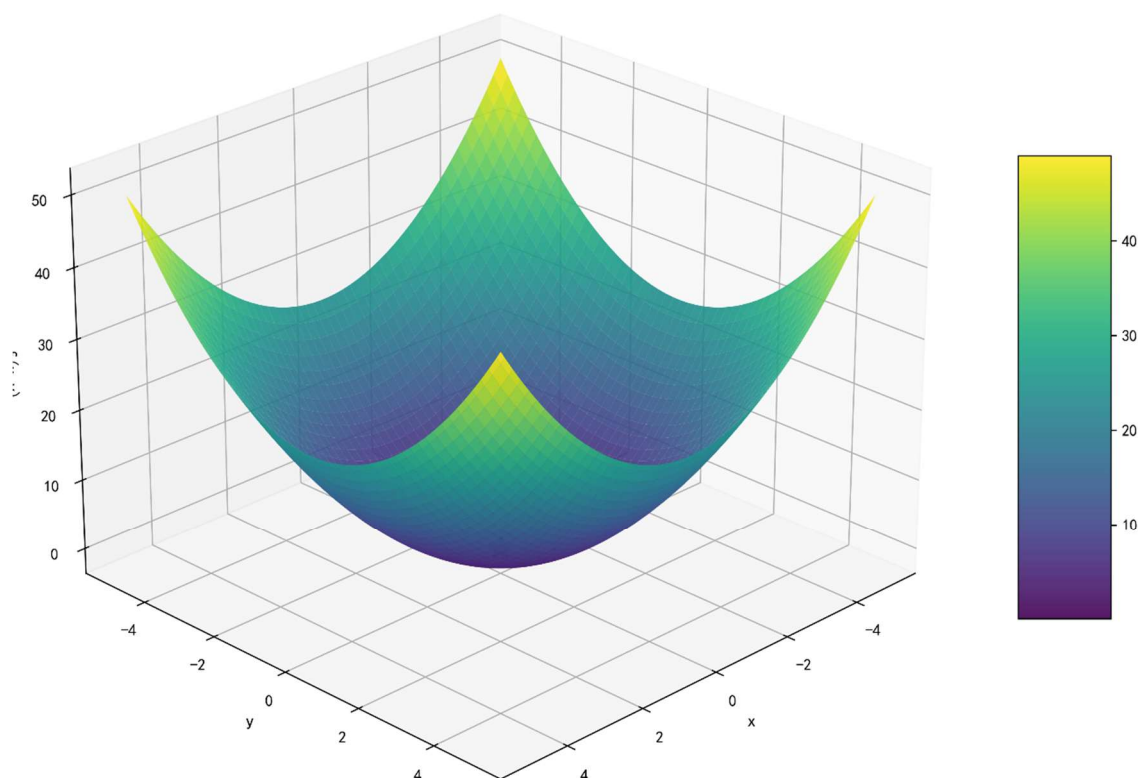


图 2 Sphere 函数 3D 图

该图为二维 Sphere 函数的 3D 可视化形态，以坐标原点为中心呈对称的“平滑球体”状，整个曲面无任何起伏或凹陷，函数值从原点向外均匀递增，能直观体现其“简单凸函数”的特征，清晰展示全局最优解（原点）的唯一性与位置确定性。

函数公式：

$$f(x) = \sum (x_i^2)$$

定义域: $[-5, 5]$ （各维度决策变量均在此区间内）

全局最优：当所有决策变量 x_i 均为 0 时，函数取得全局最小值 $f(0, \dots, 0) = 0$ 。

作为最简单的连续可微凸函数，其函数曲面平滑无起伏，最优解位置明确。该函数常作为算法测试的“入门级”案例，用于快速验证算法的基本收敛逻辑是否成立。由于其梯度信息易于计算且寻优路径清晰，Sphere 函数不仅用于 PSO 算法测试，也是各类优化算法性能对比的基准函数，能有效区分算法在无干扰情况下的收

敛速度差异。

2. Zakharov 函数（扎哈罗夫函数）

Zakharov函数 - 3D曲面图

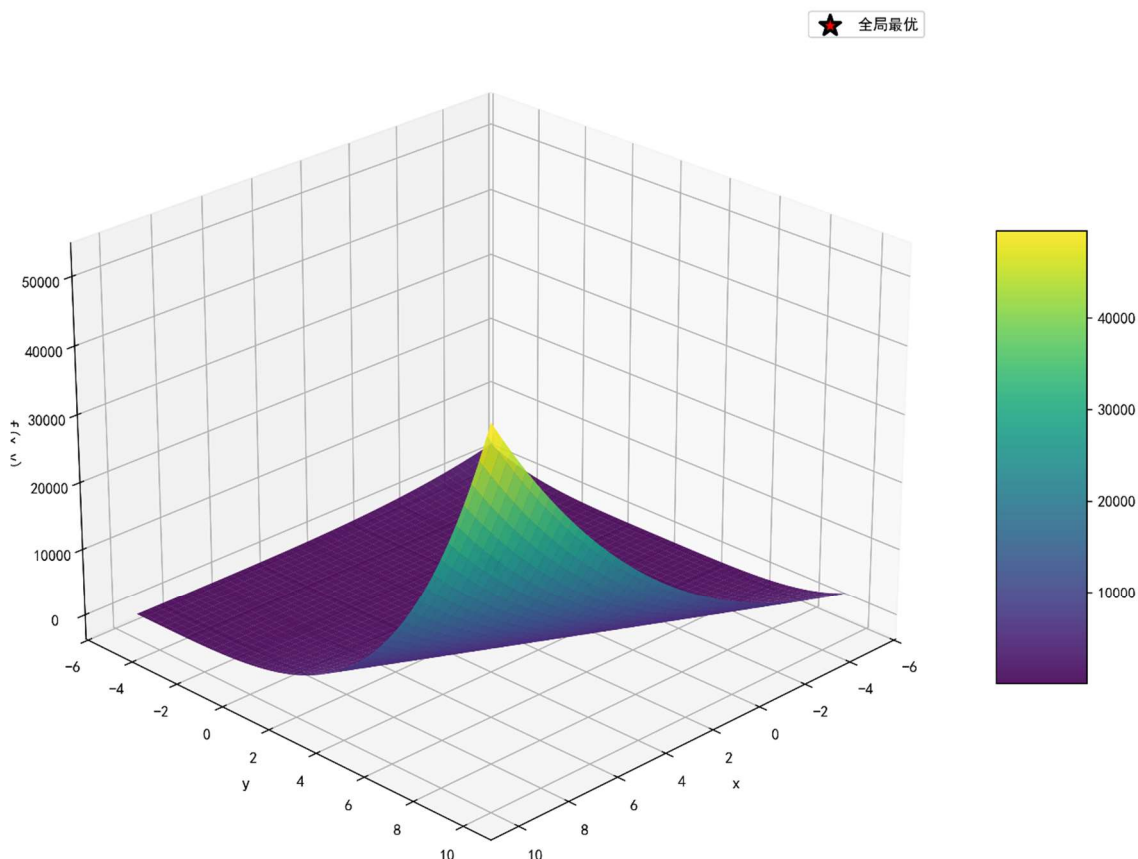


图 3 扎哈罗夫函数 3D 图

图中呈现的 Zakharov 函数 3D 形态以原点为最优解中心，相比 Sphere 函数曲面有轻微的波纹起伏，这是由函数中的交叉项导致的，但整体仍保持单峰特征，能帮助理解“变量弱关联”对函数形态的影响，以及算法需处理的轻微干扰。

函数公式：

$$f(x) = \sum (x_i^2) + (\sum (0.5 \cdot i \cdot x_i))^2 + (\sum (0.5 \cdot i \cdot x_i))^4$$

其中 i 为决策变量的维度索引， $0.5 \cdot i \cdot x_i$ 体现了维度索引对变量的加权作用。

定义域：[-5, 10]（各维度决策变量均在此区间内）

全局最优：当所有决策变量 x_i 均为 0 时，函数取得全局最小值 $f(0, \dots, 0) = 0$ 。

Zakharov 函数在 Sphere 函数的基础上引入了交叉项（含 $\sum (0.5 \cdot i \cdot x_i)$ 的平方和四次方项），使函数关系更复杂，对算法处理变量间关联信息的能力提出了一定

要求。该函数的交叉项会导致函数曲面出现轻微起伏，但仍保持单峰特性，可用于测试 PSO 算法在变量存在弱关联情况下的寻优稳定性，其难度介于 Sphere 函数与复杂单峰函数之间。

3. Dixon-Price 函数（狄克逊-普莱斯函数）

Dixon-Price函数 - 3D曲面图

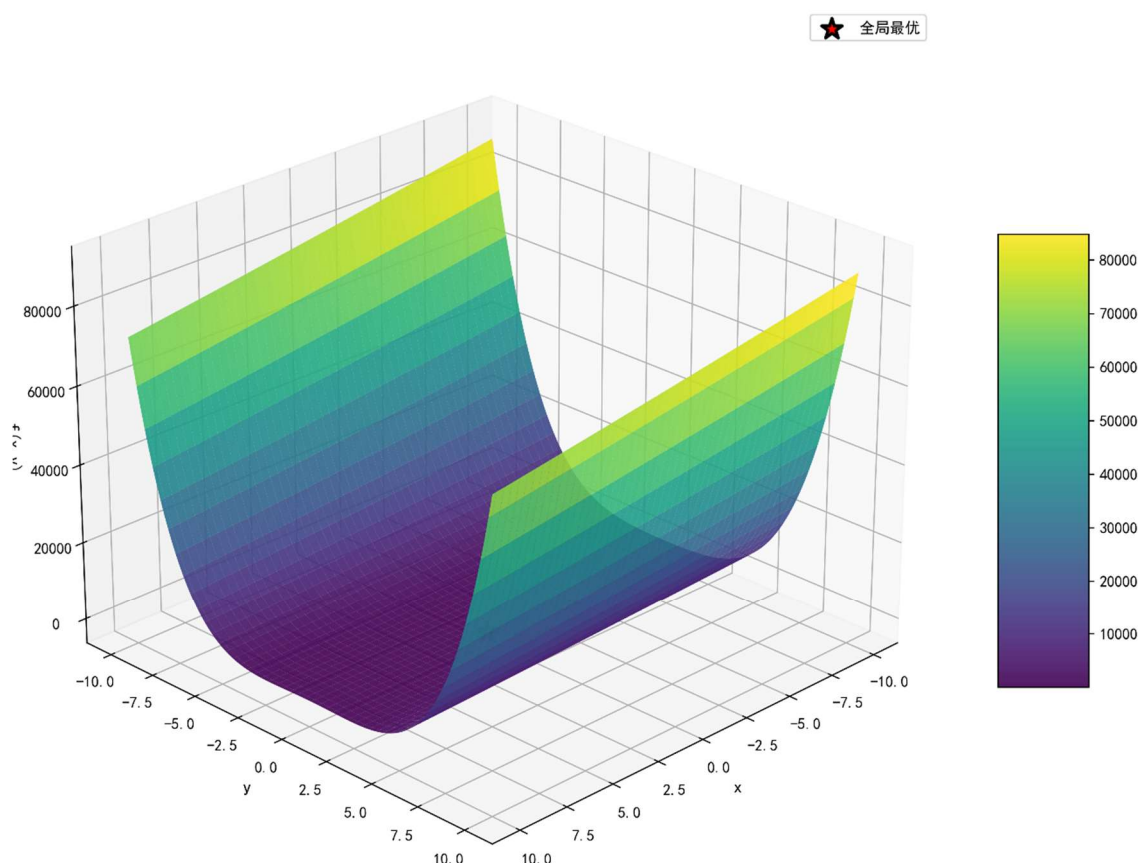


图 4 .Dixon-Price 函数 3D 图

该 3D 图核心呈现函数“对角线狭长山谷”的特征，最优解沿特定对角线分布，山谷两侧曲面平缓、底部狭窄，直观展示了算法易在两侧徘徊、难沿山谷逼近最优解的寻优难点，清晰反映其“单峰高难度”的特性。

函数公式：

$$f(x) = (x_1 - 1)^2 + \sum_{i=2}^n (i \cdot (2xi^2 - xi_{i-1}))^2$$

其中 x_1 为第一个维度的决策变量， x_i ($i \geq 2$) 为第 i 个维度的决策变量， x_{i-1} 为第 $i-1$ 个维度的决策变量。

定义域：[-10, 10]（各维度决策变量均在此区间内）

全局最优：当 $x_i = 2^{-(2^i - 2)/2^i}$ (i 为维度索引) 时，函数取得全局最小值 0。

狄克逊-普莱斯函数曲面呈现对角线形状的狭长山谷，最优解沿特定对角线分布，算法易在山谷两侧徘徊，难以快速沿山谷方向向最优解逼近。该函数的寻优难点在于需要算法精准捕捉最优解的分布规律，避免陷入山谷两侧的局部平缓区域。常用于测试 PSO 算法的方向搜索能力和步长调节的灵活性，是单峰函数中难度较高的一类。

4. Rosenbrock 函数（罗森布洛克函数）

Rosenbrock函数（香蕉函数） - 3D曲面图

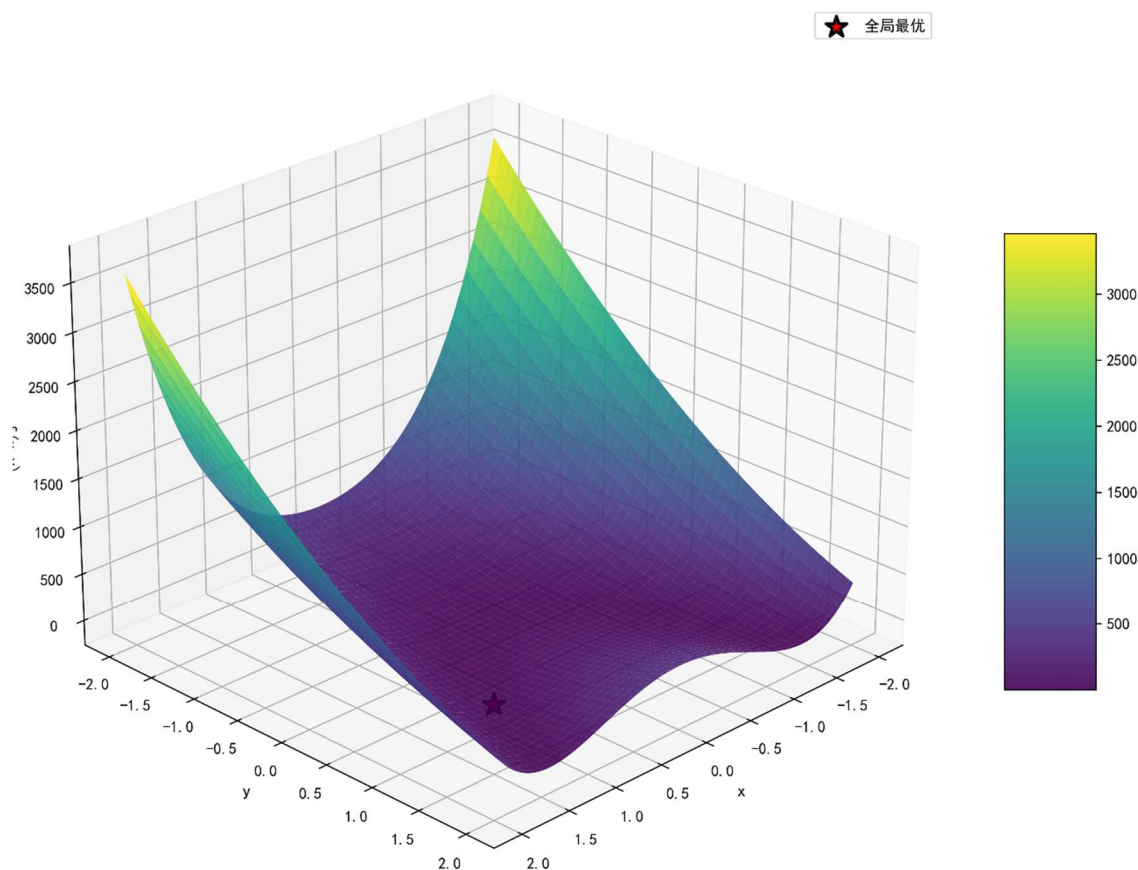


图 5 Rosenbrock 函数（罗森布洛克函数）3D 图

图中清晰呈现“香蕉形”狭长山谷的典型形态，山谷两侧曲面陡峭、底部平缓，全局最优解位于山谷末端（对应变量全为 1 的位置），能直观解释算法易被陡峭两侧误导、难定位底部最优解的核心问题，是其“迷惑性单峰”特征的直接体现。

函数公式：

$$f(x) = \sum [100 \cdot (x[i+1] - x_i)^2 + (1 - x_i)^2]$$

其中 $x[i+1]$ 为第 $i+1$ 个维度的决策变量， x_i 为第 i 个维度的决策变量，100 为加权系数。

定义域：[-2, 2]（各维度决策变量均在此区间内）

全局最优：当所有决策变量 x_i 均为 1 时，函数取得全局最小值 $f(1, \dots, 1) = 0$ 。

罗森布洛克函数曲面呈“香蕉形”狭长山谷，最优解位于山谷底部，山谷两侧陡峭但底部平缓，算法易被两侧陡峭区域误导，难以精准定位山谷底部的最优解。作为经典的“欺骗性”单峰函数，Rosenbrock 函数是测试优化算法局部搜索精度的核心基准函数。PSO 算法需通过合理调节惯性权重和学习因子，平衡全局探索与局部开发，才能高效找到山谷底部的最优解。

2.2.2 多峰函数

多峰函数存在多个局部最优解，局部最优解与全局最优解相互交织，算法易陷入局部最优而无法到达全局最优，这类函数主要用于测试算法的全局搜索能力、跳出局部最优的能力及寻优稳定性。

5. Rastrigin 函数（拉斯里金函数）

Rastrigin函数 - 3D曲面图

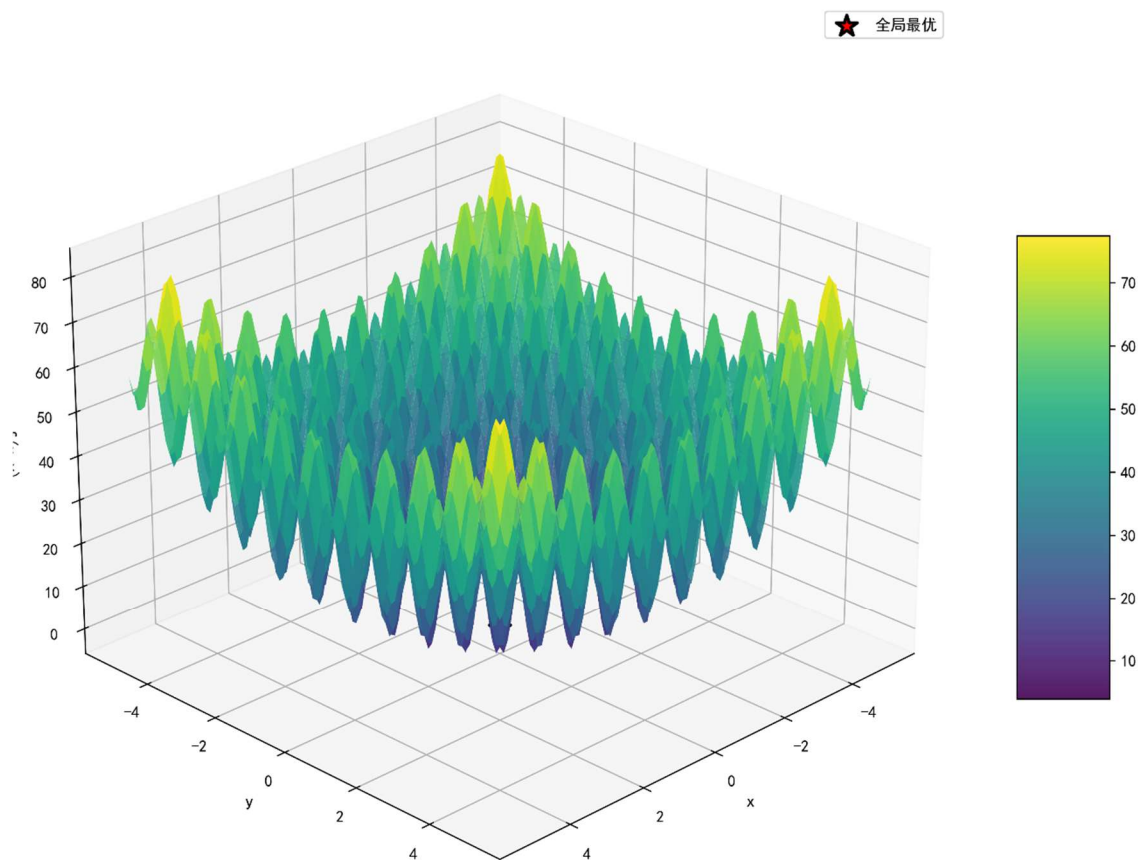


图 6 Rastrigin 函数（拉斯里金函数）3D 图

该 3D 图呈现“密集多峰”形态，以原点为全局最优解中心，整个曲面分布着大量均匀且函数值接近的局部峰，峰谷交错密集，直观展示了算法难从众多局部最优中分辨全局最优的核心挑战，清晰反映其“多峰基础测试”的定位。

函数公式：

$$f(x) = 10n + \sum [xi^2 - 10 \cdot \cos(2 \pi xi)]$$

其中 n 为函数的维度， π 为圆周率， $\cos(2 \pi xi)$ 为余弦函数。

定义域： $[-5.12, 5.12]$ （各维度决策变量均在此区间内）

全局最优：当所有决策变量 xi 均为 0 时，函数取得全局最小值 $f(0,...,0) = 0$ 。

Rastrigin 函数（拉斯里金函数）由 Sphere 函数与余弦振荡项叠加而成，函数曲面上分布着大量均匀的局部最优解，局部最优解的函数值相近，形成“多峰密布”的形态，算法极难从中识别并收敛到全局最优解。Rastrigin 函数的难度随维度增加而显著提升，是测试 PSO 算法全局搜索能力的经典函数。算法需通过较大的惯性

权重初期扩大搜索范围，再通过减小惯性权重精细筛选最优解，才能避免被局部最优“困住”。

6. Ackley 函数（阿克利函数）

Ackley函数 - 3D曲面图

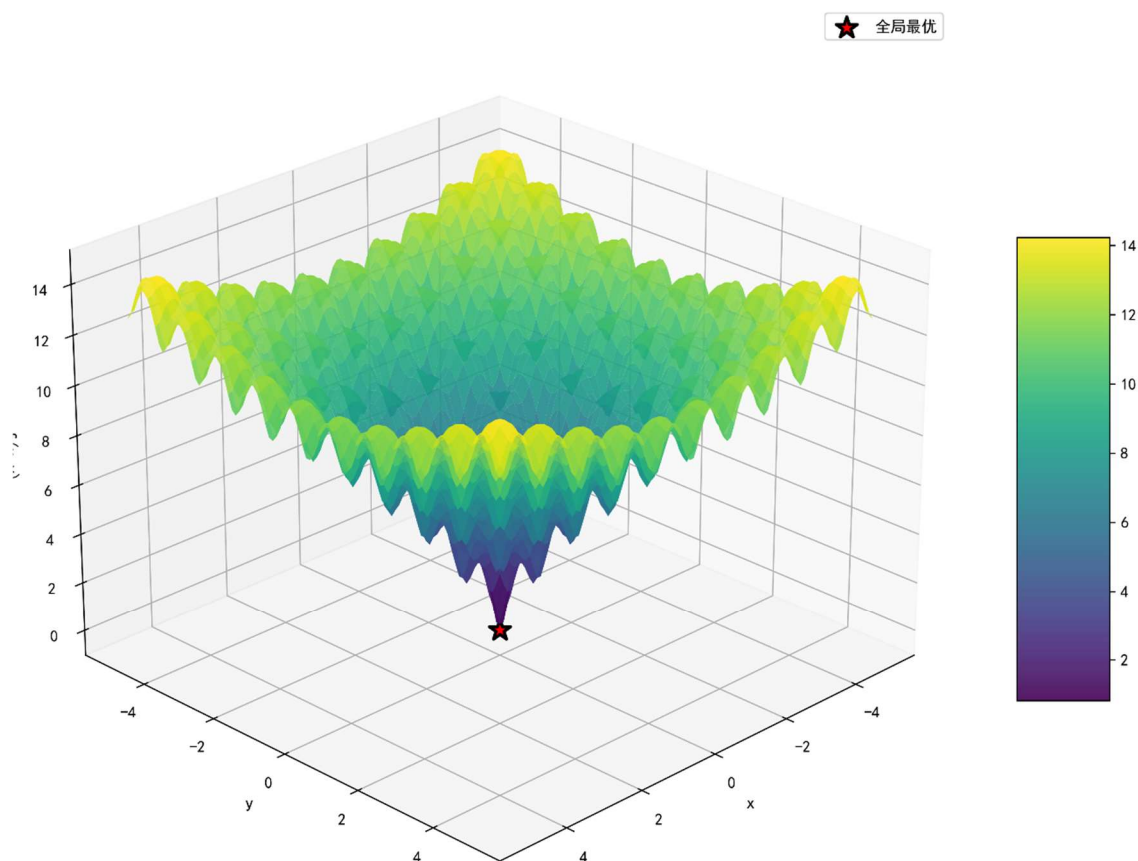


图 7 Ackley 函数 3D 图

图中核心呈现“中心深谷+外围浅峰”的结构，中心全局最优解所在深谷与外围多个局部浅峰形成鲜明对比，曲面整体平滑但峰谷层次复杂，能直观解释算法易被外围浅峰吸引、难深入中心深谷的寻优问题，体现其“平衡测试”的功能定位。

函数公式：

$$f(x) = -20 \cdot \exp(-0.2 \cdot \sqrt{(\sum x_i^2)/n}) - \exp(\sum \cos(2\pi x_i)/n) + 20 + e$$

其中 n 为函数维度， e 为自然常数（约 2.71828）， $\exp$ 为指数函数， $\sqrt{\quad}$ 为平方根函数。

定义域： $[-5, 5]$ （各维度决策变量均在此区间内）

全局最优：当所有决策变量 x_i 均为 0 时，函数取得全局最小值 $f(0, \dots, 0) = 0$ 。

Ackley 函数（阿克利函数）中心存在一个深谷（全局最优解所在位置），周围环绕着多个浅层次的局部最优解，形成“中心深谷+外围浅峰”的结构，算法易被外围局部最优吸引，难以深入中心深谷。Ackley 函数的平滑曲面和复杂峰谷结构，使其成为测试 PSO 算法“探索-开发”平衡能力的理想工具。算法需在初期快速跳过外围局部最优，同时在接近中心区域时精准调节步长，避免错过全局最优。

7. Griewank 函数（格里旺克函数）

Griewank 函数 - 3D 曲面图

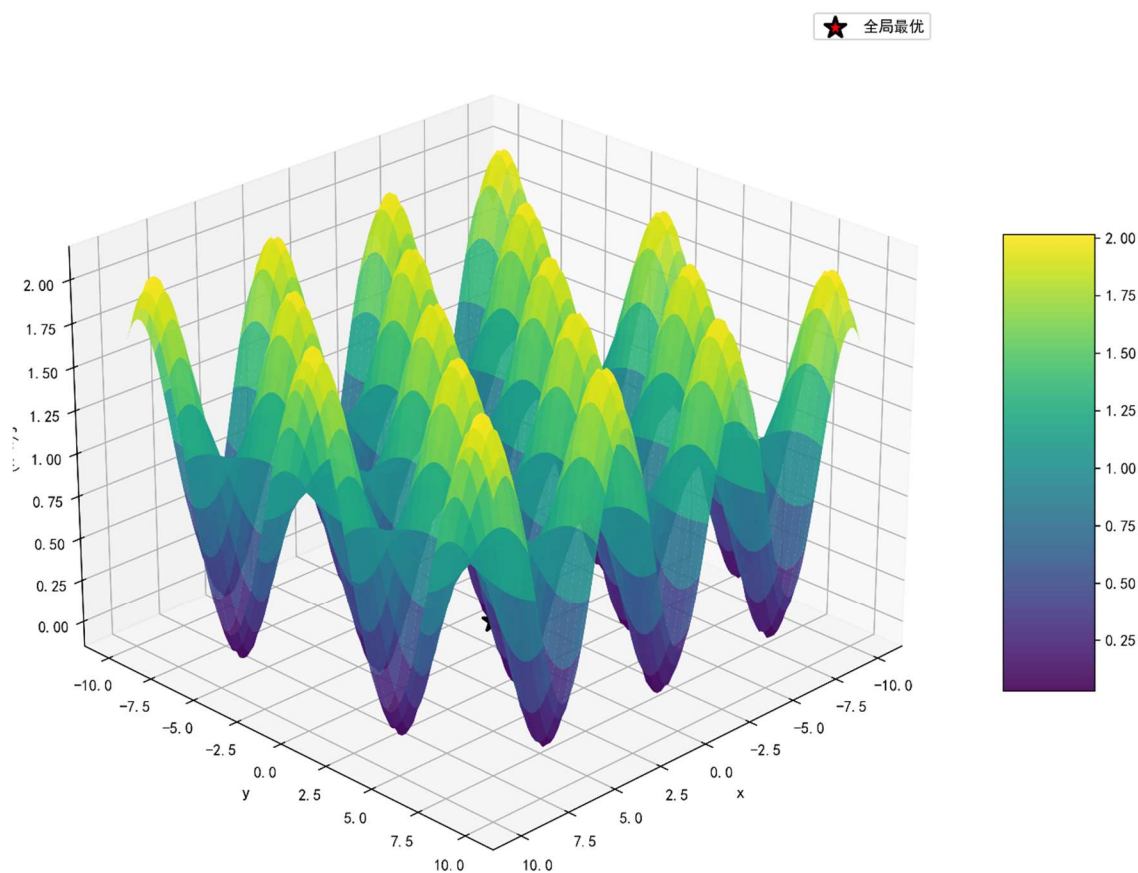


图 8 Griewank 函数 3D 图

该 3D 图展示“周期性多峰”特征，局部最优解以固定周期在曲面分布，相邻峰谷函数值差异小，随维度延伸峰谷数量递增，直观呈现了算法在大范围内需应对周期性干扰的寻优难点，反映其“高维多峰测试”的特性。

函数公式：

$$f(x) = 1 + \frac{\sum (x_i^2)/4000 - \prod (\cos(x_i/\sqrt{i}))}{4000}$$

其中 $\prod$ 表示对所有维度的 $\cos(x_i/\sqrt{i})$ 进行乘积运算， $\sqrt{i}$ 为维度索引 i 的平方

根。

定义域：[-600, 600]（各维度决策变量均在此区间内）

全局最优：当所有决策变量 x_i 均为 0 时，函数取得全局最小值 $f(0, \dots, 0) = 0$ 。

Griewank 函数（格里旺克函数）的局部最优解呈周期性分布，且相邻局部最优解的函数值差异较小，随着维度增加，局部最优解的数量大幅增多，算法的寻优难度呈指数级上升。Griewank 函数的宽定义域和周期性峰谷特性，对 PSO 算法的全局搜索范围和随机性调节提出了极高要求。算法需通过合理设置随机数参数 $r1$ 、 $r2$ ，在大范围内高效筛选最优解区域，同时避免在周期性局部最优中循环。

8. Schwefel 函数（施魏费尔函数）

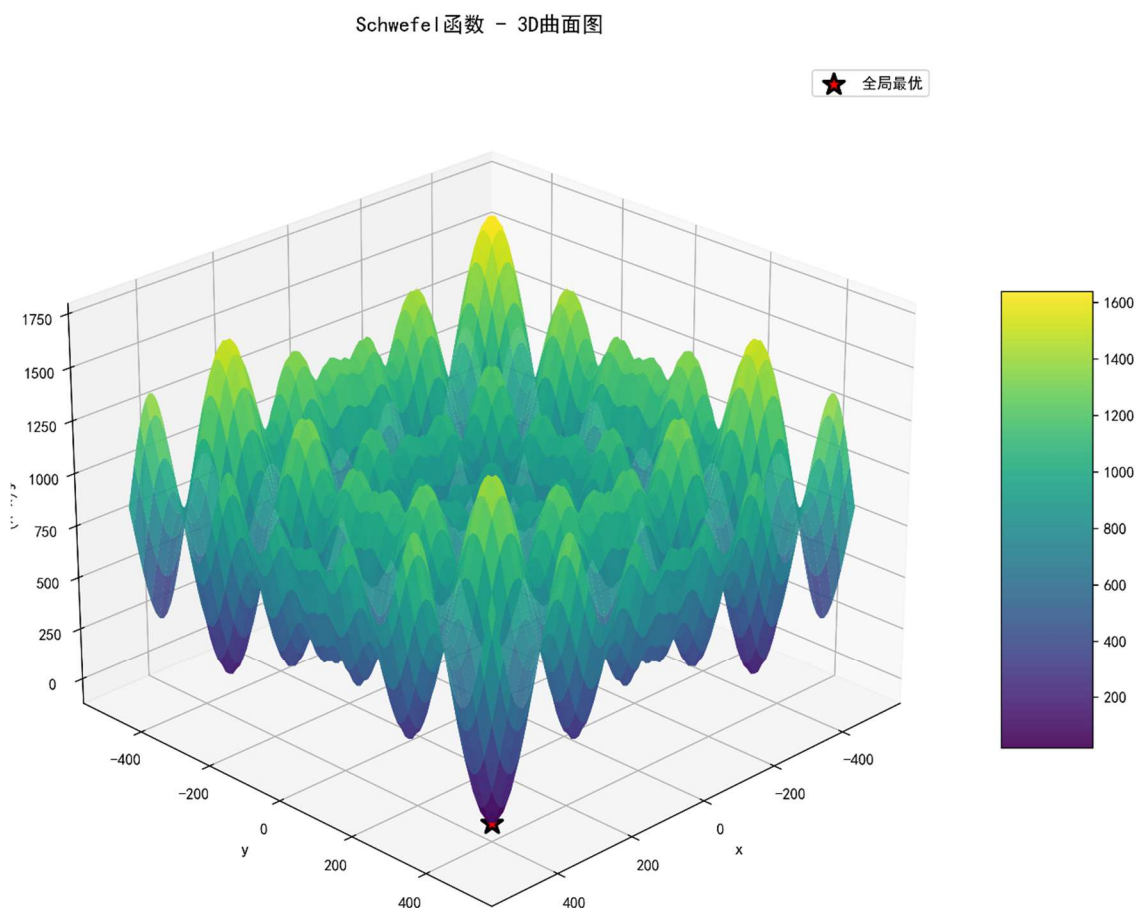


图 9 Schwefel 函数 3D 图

图中呈现“误导性多峰”形态，原点附近分布多个明显局部峰，而全局最优解位于远离原点的区域，曲面整体呈下降趋势但局部波动剧烈，直观解释了算法易被原点附近局部最优误导的核心问题，凸显其“高难度多峰”的测试价值。

函数公式：

$$f(x) = 418.9829n - \sum (x_i \cdot \sin(\sqrt{|x_i|}))$$

其中 n 为函数维度，418.9829 为归一化系数， $\sin(\sqrt{|x_i|})$ 为正弦函数与平方根函数的复合运算。

定义域：[-500, 500]（各维度决策变量均在此区间内）

全局最优：当所有决策变量 x_i 均为 420.9687 时，函数取得近似全局最小值 $f(420.9687, \dots, 420.9687) \approx 0$ 。

Schwefel 函数(施魏费尔函数)具有极强的欺骗性,全局最优解远离坐标原点,而原点附近存在多个局部最优解,算法易被原点附近的局部最优“迷惑”,难以向远处的全局最优解收敛。Schwefel 函数的函数值整体呈下降趋势,但其局部波动剧烈,是测试 PSO 算法全局探索能力和跳出局部最优能力的“硬核”基准函数。算法需在初期保持较大的惯性权重,扩大搜索范围,才能有机会发现远离原点的全局最优区域。

9. Levy 函数（利维函数）

Levy函数 - 3D曲面图

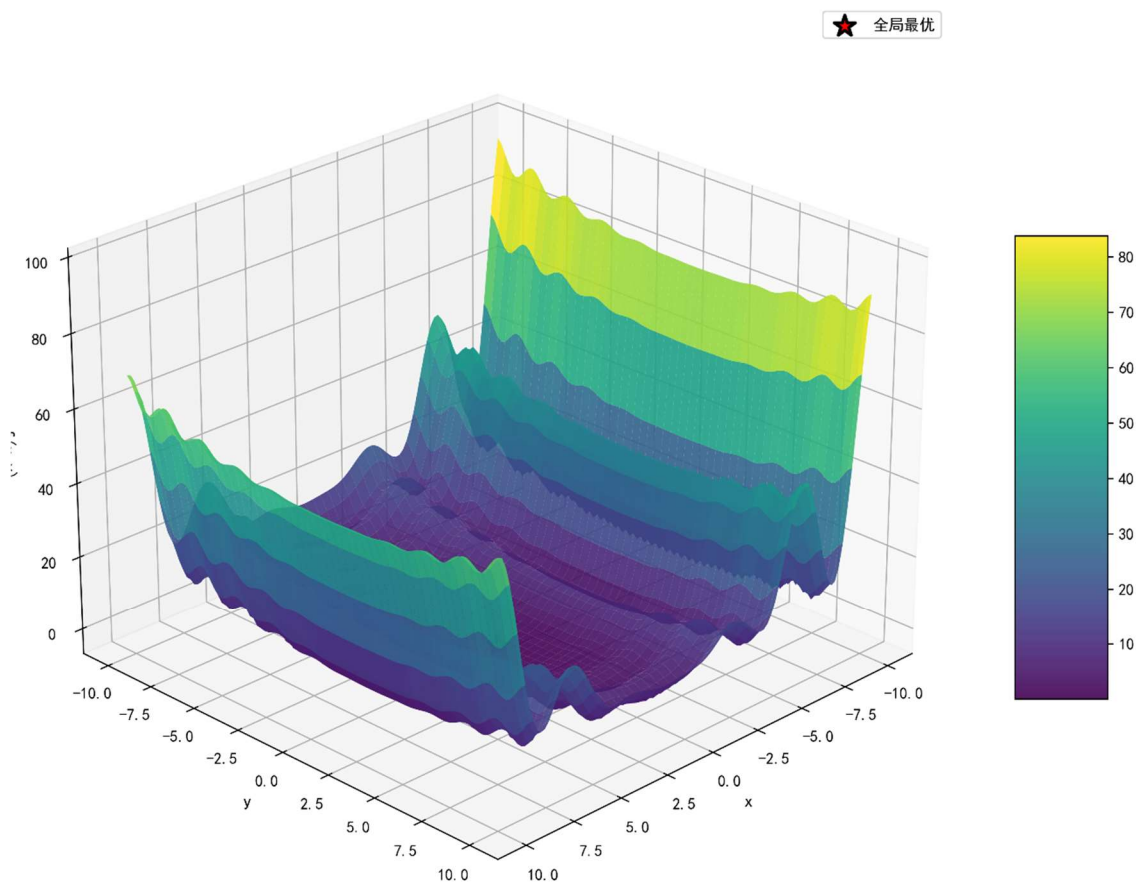


图 10 Levy 函数 3D 图

该 3D 图展示“不规则多峰”结构，局部最优解分布密度不均，部分区域峰谷密集、部分区域平缓开阔，曲面形态无明显规律，直观反映了算法需自适应调整搜索策略的寻优需求，体现其“复杂解空间测试”的定位。

函数公式：

$$f(x) = \sin(\pi w_1)^2 + \sum [(w_i - 1)^2 \cdot (1 + 10 \cdot \sin^2(\pi w_1 + 1))] + (w_n - 1)^2 \cdot (1 + \sin^2(2\pi w_n))$$

其中 $w_i = 1 + (x_i - 1)/4$ ， x_i 为决策变量。

定义域：[-10, 10]（各维度决策变量均在此区间内）

全局最优：当所有决策变量 x_i 均为 1 时，函数取得全局最小值 $f(1, \dots, 1) = 0$ 。

Levy 函数曲面呈不规则多峰形态，局部最优解的分布密度不均，部分区域峰谷密集，部分区域相对平缓，对算法的自适应搜索能力要求较高。Levy 函数的复杂峰谷结构模拟了实际优化问题中的非均匀解空间，PSO 算法需通过动态调整惯性权重和学习因子，在峰谷密集区域精细搜索，在平缓区域快速探索，才能高效找

到全局最优解。

10. Michalewicz 函数（米哈列维奇函数）

Michalewicz函数 - 3D曲面图

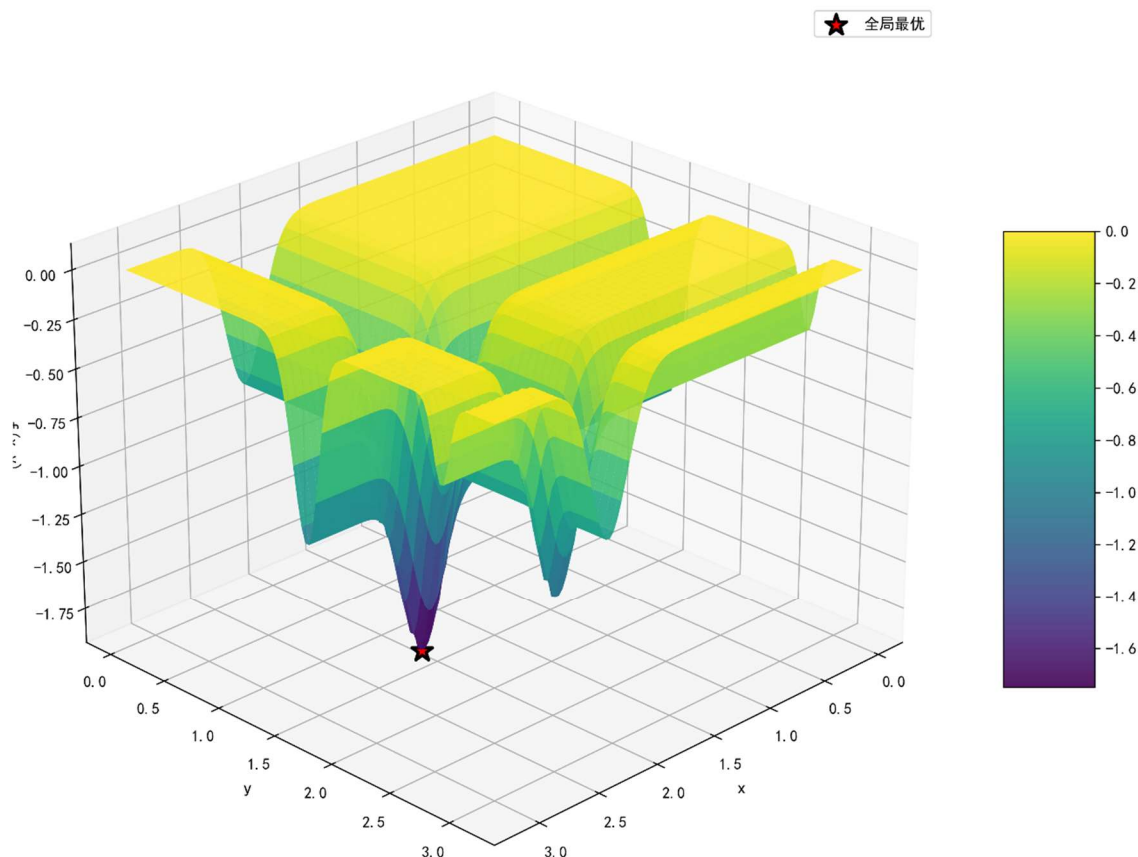


图 11 Michalewicz 函数 3D 图

图中呈现“维度相关多峰”特征，二维时峰谷数量较少，随维度提升峰谷呈指数级增多，最终形成“山峰海洋”形态，直观展示了高维场景下的寻优难度，清晰反映其“高维性能评估”的核心作用。

函数公式：

$$f(x) = -\sum (\sin(x_i) \cdot \sin^{2m}(i \cdot x_i^2 / \pi))$$

其中 $m=10$ （常用取值）， i 为维度索引， π 为圆周率， $\sin^{2m}$ 表示正弦函数的 $2m$ 次幂。

定义域： $[0, \pi]$ （各维度决策变量均在此区间内）

全局最优：二维情况下，函数全局最优值约为-1.8013，高维情况下最优值随维度变化而降低。

Michalewicz 函数（米哈列维奇函数）的峰数量随维度增加而显著增多，二维时仅有少数几个峰，三维时峰数量大幅增加，高维情况下形成“峰海”形态，算法极难从中识别全局最优。Michalewicz 函数是测试高维优化问题中 PSO 算法性能的重要函数，其峰数量与维度的强关联特性，能有效考核算法在高维复杂解空间中的寻优效率，常用于评估改进型 PSO 算法的性能提升效果。

11. Styblinski-Tang 函数（斯泰布林斯基-唐函数）

Styblinski-Tang函数 - 3D曲面图

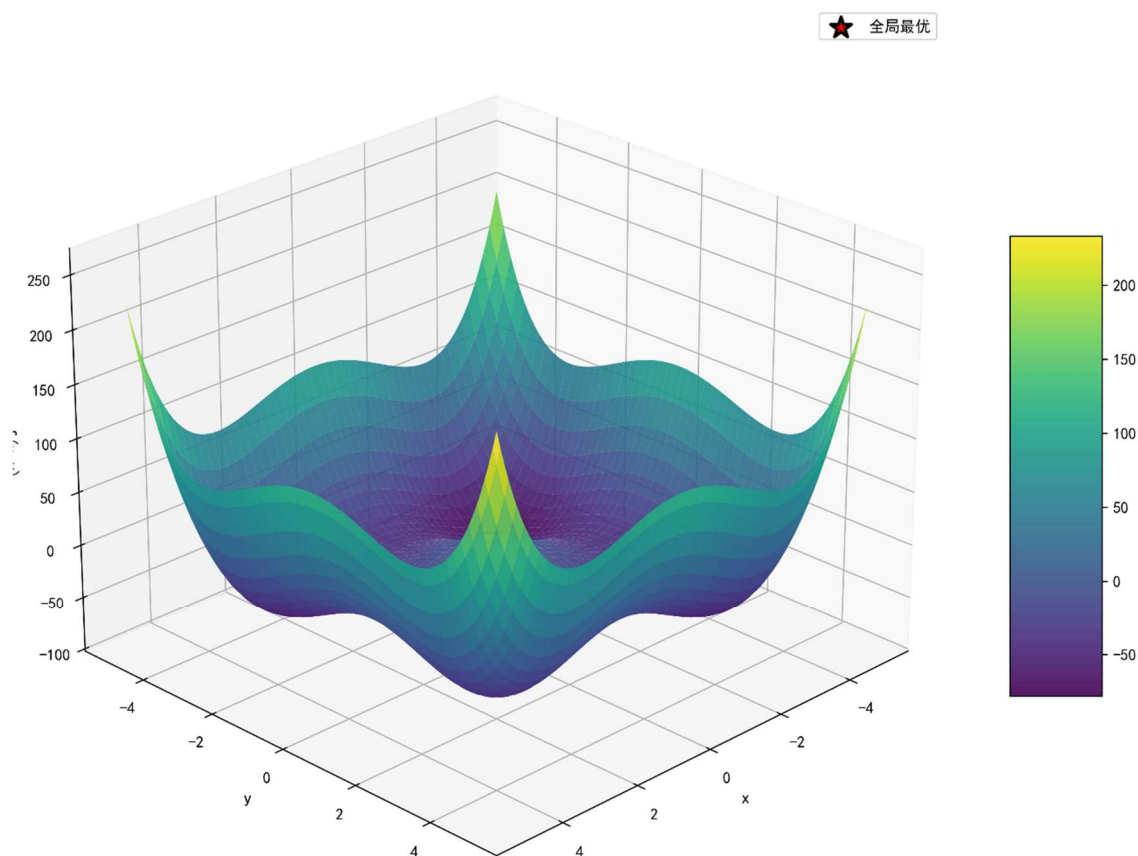


图 12 Styblinski-Tang 函数 3D 图

该 3D 图展示“对称多峰”形态，多个局部最优解对称分布，全局最优解（负值）与局部最优解函数值差异明显，曲面由多项式结构形成平滑峰谷，直观体现了算法可通过函数值对比定位最优方向的特点，反映其“易用性多峰测试”的优势。

函数公式：

$$f(x) = \sum (xi^4 - 16xi^2 + 5xi) / 2$$

其中 xi 为决策变量， $\sum$ 表示对所有维度的决策变量进行求和运算后再除以 2。

定义域: $[-5, 5]$ (各维度决策变量均在此区间内)

全局最优: 当所有决策变量 x_i 均为 -2.903534 时, 函数取得全局最小值 $f(-2.903534, \dots, -2.903534) \approx -39.16599 \cdot n$ (n 为维度)。

Styblinski-Tang 函数 (斯泰布林斯基-唐函数) 全局最优解为负值, 函数曲面存在多个对称分布的局部最优解, 局部最优与全局最优的函数值差异明显, 算法可通过函数值对比初步判断最优解方向。: tyblinski-Tang 函数的多项式结构使其函数值计算简便, 同时多峰特性又能有效测试算法性能, 是兼顾易用性和测试有效性的多峰函数。PSO 算法可通过对比粒子的个体最优和全局最优值, 快速向函数值更低的全局最优区域收敛。

2.3 实际问题-背包问题

为验证粒子群优化算法在实际组合优化问题中的性能, 本节选取经典的 0-1 背包问题作为研究对象, 并引入动态规划算法作为性能对比基准。0-1 背包问题是资源分配类问题的典型代表, 其求解逻辑具有较强的通用性, 在物流配装、资金规划等实际场景中应用广泛。

2.3.1 问题定义

选取背包问题作为解决的实际问题。也就是给定 n 个物品和一个容量为 C 的背包, 有:

- 每个物品 i 有重量 $w[i]$ 和价值 $v[i]$
- 每个物品只能选择装或不装 (0-1 约束)
- 目标: 在不超过背包容量的前提下, 使装入物品的总价值最大

为清晰界定问题边界与目标, 建立如下数学模型。其中引入决策变量 $x[i]$ 表示第 i 个物品的选择状态, $x[i] \in \{0, 1\}$; 目标函数用于刻画“总价值最大化”的核心需求, 约束条件则明确了“重量不超限”和“0-1 选择”两大限制:

- 目标函数 (最大化): $\max f(x) = \sum (v[i] \cdot x[i])$ (i 从 1 到 n 求和)

- 约束条件:

(1) 重量约束: $\sum (w[i] \cdot x[i]) \leq C$ (i 从 1 到 n 求和, 确保总重量不超过背包容量)

(2) 0-1 约束: $x[i] \in \{0, 1\}$, $i = 1, 2, \dots, n$ (每个物品仅有两种选择)

2.3.2 动态规划算法

2.3.2 动态规划算法

动态规划算法是求解 0-1 背包问题的经典有效方法，其核心思想是将原问题拆解为多个相互关联的子问题，通过存储子问题的最优解避免重复计算，从而降低求解复杂度。该算法适用于具有“最优子结构”和“重叠子问题”特性的问题，0-1 背包问题恰好满足这两个条件——子问题的最优解可构成原问题的最优解，且不同子问题之间存在大量重复计算。

1. 状态定义：

动态规划算法的关键是定义合理的状态以描述子问题。本节定义二维状态数组 $dp[i][w]$ ，其核心含义为“前 i 个物品在背包容量为 w 时，能够装入的物品的最大价值”。该状态定义将原问题（ n 个物品、容量 C ）拆解为“前 i 个物品、容量 w ”的子问题，通过逐步求解子问题实现对原问题的解答。

2. 状态转移方程：

状态转移方程反映了子问题之间的递推关系，其本质是对“是否装入第 i 个物品”这一决策进行枚举，并取两种决策下的最优结果。具体递推逻辑如下：

当考虑第 i 个物品时，存在两种选择：

(1) 不装入第 i 个物品：此时背包的容量仍为 w ，前 i 个物品的最大价值等价于前 $i-1$ 个物品在容量 w 时的最大价值，即 $dp[i][w] = dp[i-1][w]$ ；

(2) 装入第 i 个物品：该决策的前提是背包剩余容量能够容纳第 i 个物品（即 $w \geq w[i]$ ），此时前 i 个物品的最大价值为“前 $i-1$ 个物品在容量 $w - w[i]$ 时的最大价值”加上第 i 个物品的价值，即 $dp[i][w] = dp[i-1][w - w[i]] + v[i]$ 。

综合两种决策，状态转移方程为：

$$dp[i][w] = \max\{dp[i-1][w], dp[i-1][w - w[i]] + v[i]\} \quad (\text{当 } w \geq w[i] \text{ 时})$$

若 $w < w[i]$ ，则无法装入第 i 个物品，此时 $dp[i][w] = dp[i-1][w]$ 。

3. 边界条件：

边界条件用于确定子问题的初始状态，为递推计算提供起点，具体包括以下两种情况：

(1) 当物品数量为 0 时（ $i=0$ ）：无论背包容量 w 为多少，均无物品可装入，因此最大价值为 0，即 $dp[0][w] = 0$ （ $w \in [0, C]$ ）；

(2) 当背包容量为 0 时（ $w=0$ ）：无论物品数量 i 为多少，均无法装入任何物

品，因此最大价值为 0，即 $dp[i][0] = 0$ ($i \in [0, n]$)。

三、系统架构

为将粒子群优化算法（PSO）及动态规划算法的理论模型转化为可交互的实用工具，本节从系统架构、核心模块、数据库设计及用户界面四个维度，详细阐述系统的设计思路与实现要点。系统整体采用分层架构设计，确保各模块职责清晰、耦合度低，便于后续功能扩展与维护。

3.1 系统架构

系统采用“三层架构+模块化”的设计方式，从上到下分为用户界面层、算法层和数据访问层，每层内部按功能拆成独立模块，形成“模块内部功能集中，模块之间关联性低”的架构体系，具体架构如下图所示：

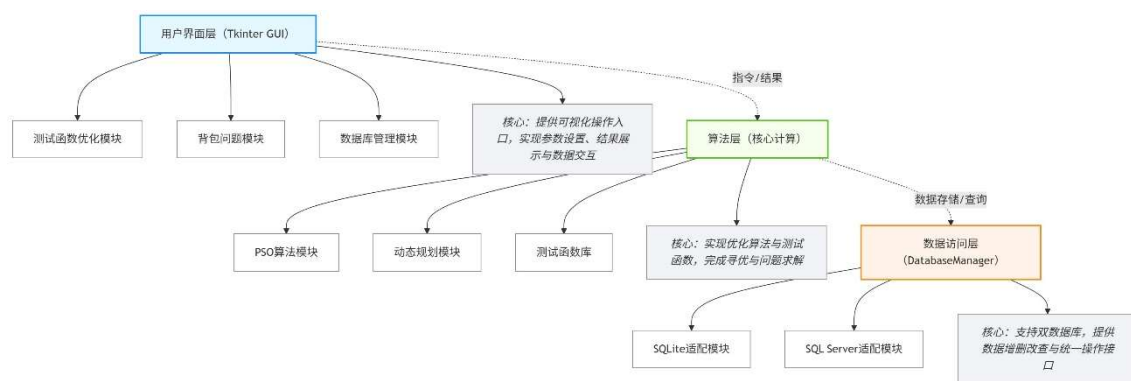


图 13 系统架构图

各层之间通过接口传递数据：用户界面层接收用户的操作指令并传给算法层，算法层计算完成后把结果返回给界面层展示，同时调用数据访问层的接口把重要数据保存起来；数据访问层屏蔽了不同数据库的操作差异，给上层模块提供统一的数据库操作方式。

3.2 核心模块

核心模块是实现系统功能的核心部分，包括数据管理、算法实现、问题求解等关键功能，每个模块的设计和实现直接影响系统的性能和使用体验。

3.2.1 数据库管理模块

数据库管理模块是系统数据持久化的核心，通过封装统一的数据库操作接口，支持 SQLite 与 SQL Server 双数据库环境，满足不同场景下的数据存储需求，同时

提供自动化的数据库初始化与灵活的数据操作功能。

实现此功能的核心代码为：

```
> def connect_sqlserver(self, server, database, username, password): ...  
> def save_test_result(self, func_name, best_value, best_position, history, params): ...  
> def save_knapsack_result(self, max_value, selected_items, total_weight, history, params): ...  
> def load_test_results(self, limit=10): ...
```

图 14 数据库功能核心代码框架

此功能内置 SQLite 与 SQL Server 两种数据库的适配逻辑，用户可根据需求选择对应数据库，模块自动适配操作语法；系统启动时自动检测数据库连接状态，若为新数据库则执行建表语句完成初始化，无需手动配置；提供测试函数优化结果、背包问题求解结果的增删改查接口，支持单条数据操作与批量数据处理；支持在系统运行过程中切换数据库，切换后自动沿用当前数据操作上下文，保证数据一致性。

3.2.2 PSO 算法模块

PSO 算法模块是系统的核心计算单元，基于前文 2.1 节的 PSO 算法理论模型，实现标准化的算法逻辑，同时支持参数自定义配置，满足不同测试场景下的算法性能调优需求，并记录算法迭代过程中的收敛历史，为后续结果分析提供数据支撑。

此模块严格遵循 PSO 算法的速度与位置更新公式，实现粒子初始化、适应度评估、最优解更新等完整流程；支持用户对粒子维度、搜索范围、最大迭代次数等关键参数进行配置，适配不同测试函数的寻优需求；实时记录每轮迭代的全局最优值，生成收敛历史数据，用于后续绘制收敛曲线图；迭代结束后输出全局最优位置、最优值及收敛历史，为界面展示与数据存储提供支持。

3.2.3 测试函数模块

测试函数模块是 PSO 算法的性能测试载体，基于前文 2.2 节的 11 种标准测试函数（4 种单峰函数、7 种多峰函数），实现函数的计算逻辑与信息封装，为算法模块提供统一的函数调用接口。其核心功能为：精准复现 11 种测试函数的数学公式，确保函数值计算的准确性；为每个函数关联定义域、全局最优值、函数特点等信息，便于用户查询与场景选择；提供标准化的函数调用方法，输入粒子位置即可返回对应的适应度值，降低与算法模块的耦合度；按单峰、多峰函数对 11 种测试函数进行分类，方便用户根据测试目标快速筛选。

3.2.4 背包问题求解模块

背包问题求解模块是算法层与实际问题的结合点，基于前文 2.3 节的 0-1 背包问题模型，实现动态规划求解逻辑，同时增加结果回溯与过程记录功能，提升问题求解的完整性与可分析性。此模块按照动态规划算法的状态定义、转移方程与边界条件，计算背包问题的最大价值；通过反向遍历动态规划状态数组，回溯得出最优解对应的选中物品集合，明确具体的物品选择方案；记录动态规划算法执行过程中的状态数组变化，生成优化历史数据，用于界面展示优化过程；支持用户自定义背包容量、物品数量及每个物品的重量与价值，适配不同的背包问题场景。

3.3 数据库设计

数据库设计遵循“按需存储、便于查询”的原则，针对测试函数优化与背包问题求解两类核心任务，设计两个独立的数据表，分别存储对应的求解结果，同时采用 JSON 格式存储收敛历史、算法参数等复杂数据，兼顾数据存储的规范性与灵活性。也就是说，使用 sqlsetuo.sql 来创建了本实验需要的数据库以及表。

3.3.1 测试函数结果表

该表用于存储 PSO 算法优化 11 种测试函数的所有结果数据，便于用户后续查询历史优化记录、对比不同参数下的算法性能。表结构及字段说明如下：

```
CREATE TABLE test_function_results (  
    id INT IDENTITY(1,1) PRIMARY KEY,          -- 自增主键  
    func_name NVARCHAR(100) NOT NULL,           -- 函数名称 (如: Sphere, Rastrigin)  
    timestamp NVARCHAR(50) NOT NULL,            -- 求解时间  
    best_value FLOAT NOT NULL,                   -- 最优值 (适应度值)  
    best_position NVARCHAR(MAX),                 -- 最优位置 (JSON格式存储坐标)  
    convergence_history NVARCHAR(MAX),          -- 收敛历史 (JSON格式)  
    algorithm_params NVARCHAR(MAX),             -- 算法参数 (JSON格式)  
    created_at DATETIME DEFAULT GETDATE()       -- 记录创建时间  
);  
GO
```

图 15 测试函数结果表

3.3.2 背包问题结果表

该表用于存储动态规划算法求解 0-1 背包问题的结果数据，重点记录最优解的核心信息（最大价值、选中物品等），支持用户回溯具体的求解方案。表结构及字段说明如下：

```
CREATE TABLE knapsack_results (
    id INT IDENTITY(1,1) PRIMARY KEY,
    timestamp NVARCHAR(50) NOT NULL,
    max_value INT NOT NULL,
    selected_items NVARCHAR(MAX),
    total_weight INT NOT NULL,
    optimization_history NVARCHAR(MAX),
    problem_params NVARCHAR(MAX),
    created_at DATETIME DEFAULT GETDATE()
);
```

图 16 背包问题结果表

3.4 用户界面设计

用户界面基于 Tkinter 框架开发，采用“多标签页+模块化布局”的设计思路，确保界面操作直观、信息展示清晰，降低用户的使用门槛，同时集成 Matplotlib 可视化组件，实现优化结果的图形化展示。

3.4.1 主界面布局

主界面采用标签页（Notebook）组件实现功能分区，共包含四个核心标签页，每个标签页内部按“操作区+结果区”的逻辑布局，具体设计如下：

1. 测试函数优化页：左侧为“参数控制区”，包含测试函数选择下拉框、PSO 算法参数输入框、开始/暂停按钮；右侧为“结果展示区”，分为数据面板（显示最优值、最优位置等）与图形面板（展示收敛曲线图）；

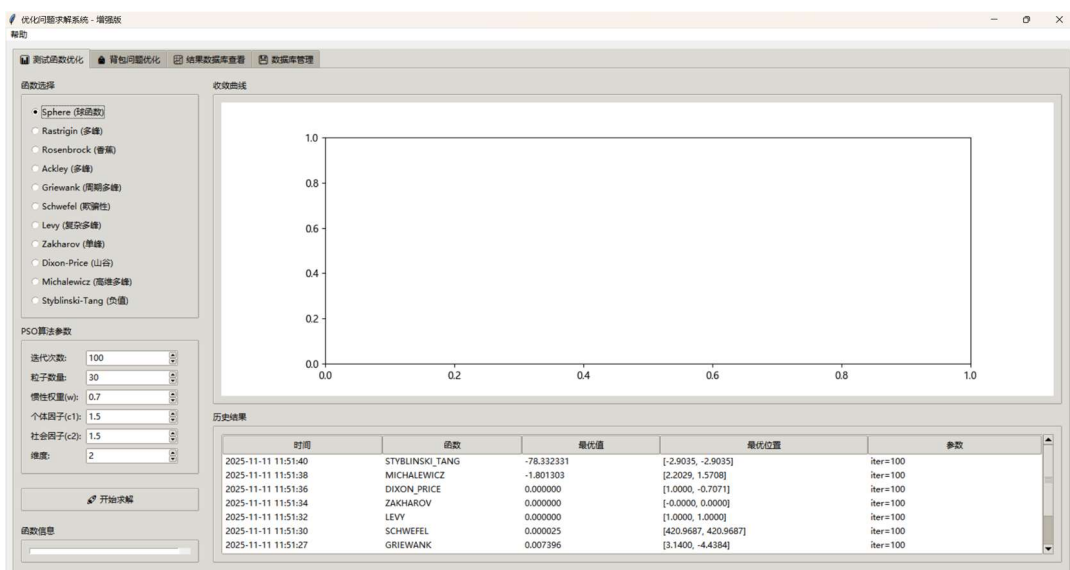


图 17 测试系统优化页

2. 背包问题页：左侧为“物品管理区”，支持手动输入或批量导入物品的重量与价值，以及设置背包容量；右侧为“优化结果区”，展示最大价值、总重量、选中物品列表及动态规划优化过程图；

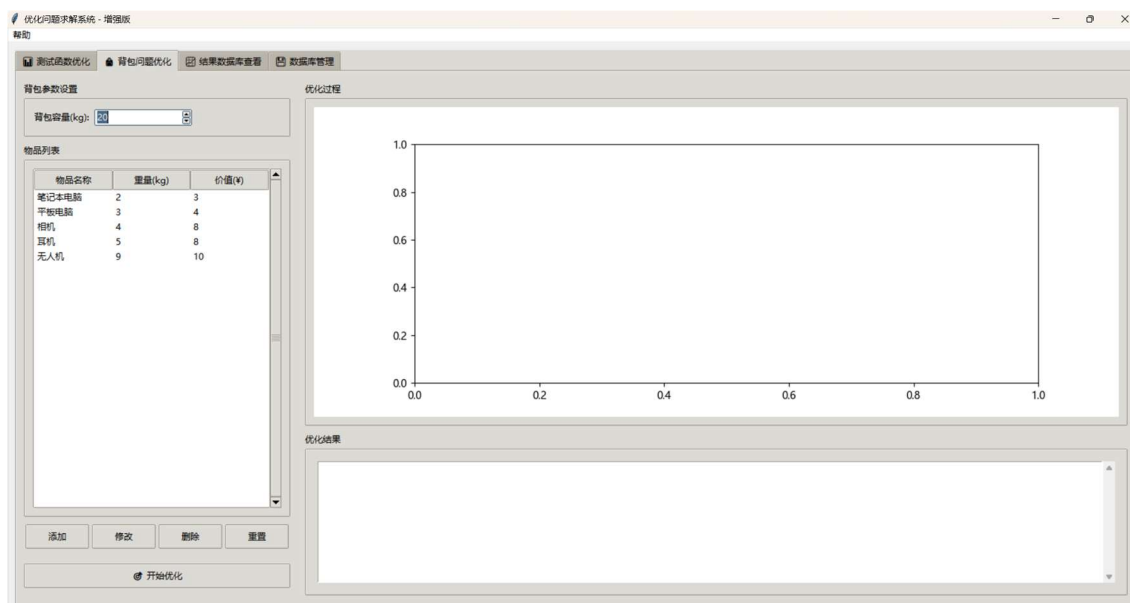


图 18 背包问题页

3. 结果查看页面：左侧为“结果筛选区”，包含结果类型切换（测试函数优化/背包问题求解）、时间范围选择、关键参数筛选（如 PSO 粒子数量、背包容量）及查询按钮；右侧为“综合展示区”，上方是“结果概览面板”（显示符合条件的结果总数、最优结果 TOP3），中间是“详细数据表格”（展示结果核心信息，支持点击行展开完整参数），下方是“多结果对比图”（可勾选多条结果，通过折线图对比收敛速度或价值差异）；

优化问题求解系统 - 增强版

帮助

测试函数优化 背包问题优化 结果数据库查看 数据库管理

数据表选择: 测试函数结果 背包问题结果 刷新 清空所有数据 查看统计

测试函数优化结果数据库

ID	函数名称	时间戳	最优值	最优位置	算法参数
15	STYBLINSKI_TANG	2025-11-11 11:51:40	-78.33233141	[-2.9035, -2.9035]	Iter=100, size=30, w=0.7
14	MICHALEWICZ	2025-11-11 11:51:38	-1.80130341	[2.2029, 1.5708]	Iter=100, size=30, w=0.7
13	DIXON_PRICE	2025-11-11 11:51:36	0.00000000	[1.0000, -0.7071]	Iter=100, size=30, w=0.7
12	ZAKHAROV	2025-11-11 11:51:34	0.00000000	[-0.0000, 0.0000]	Iter=100, size=30, w=0.7
11	LEVY	2025-11-11 11:51:32	0.00000000	[1.0000, 1.0000]	Iter=100, size=30, w=0.7
10	SCHWELF	2025-11-11 11:51:30	0.00002546	[420.9687, 420.9687]	Iter=100, size=30, w=0.7
9	GRIEWANK	2025-11-11 11:51:27	0.00739604	[3.1400, -4.4384]	Iter=100, size=30, w=0.7
8	ACKLEY	2025-11-11 11:51:25	0.00000049	[-0.0000, 0.0000]	Iter=100, size=30, w=0.7
7	ROSENBRACK	2025-11-11 11:51:23	0.00000000	[1.0000, 1.0000]	Iter=100, size=30, w=0.7
6	RASTRIGIN	2025-11-11 11:51:21	0.99495906	[0.9950, 0.0000]	Iter=100, size=30, w=0.7
5	SPHERE	2025-11-11 11:51:18	0.00000000	[-0.0000, 0.0000]	Iter=100, size=30, w=0.7
4	SPHERE	2025-11-11 11:51:14	0.00000000	[0.0000, 0.0000]	Iter=1000, size=30, w=0.7
3	SPHERE	2025-11-10 13:43:45	0.00000000	[0.0000, -0.0000]	Iter=1000, size=30, w=0.7
2	SPHERE	2025-11-10 13:43:40	0.00000000	[0.0000, -0.0000]	Iter=100, size=30, w=0.7
1	SPHERE	2024-01-01 10:00:00	0.00123000	[0.0234, -0.0156]	Iter=100, size=30, w=0.7

测试函数结果记录数: 15

图 19 结果查看页面

4. 数据库管理页：左侧为“数据库配置区”，提供 SQLite 与 SQL Server 的连接选项，SQL Server 需输入服务器、数据库名、账号密码等参数；右侧为“数据管理区”，展示历史结果数据表格，支持按时间、函数名等条件筛选，以及数据的删除与导出。

优化问题求解系统 - 增强版

帮助

测试函数优化 背包问题优化 结果数据库查看 数据库管理

数据库状态

当前使用: SQL Server (localhost)
状态: 已连接

SQL Server 连接配置

服务器地址: localhost

数据库名称: OptimizationDB

用户名: sa

密码: *****

连接 SQL Server 切换到 SQLite

使用说明

数据库使用说明:

1. SQLite 模式 (默认):
 - 无需配置, 自动创建本地数据库文件
 - 数据保存在 optimization_results.db 文件中
 - 适合个人使用和测试
2. SQL Server 模式:
 - 需要先安装 SQL Server
 - 需要安装 pyodbc 库: pip install pyodbc
 - 填写服务器信息后点击“连接 SQL Server”
 - 适合团队使用和大数据量存储

© 2024 SASEE

图 20 数据库管理页面

3.4.2 可视化组件

可视化组件是提升用户体验的核心，通过图形化方式直观呈现算法性能与求解过程，主要包含三类核心组件：

1. 收敛曲线图: 基于 Matplotlib 绘制, 以迭代次数为横轴、全局最优值为纵轴, 实时展示 PSO 算法的收敛过程, 帮助用户快速判断算法的收敛速度与稳定性;
2. 优化过程图: 针对背包问题设计, 以背包容量为横轴、最大价值为纵轴, 展示动态规划状态数组的变化曲线, 直观呈现最优解的形成过程;
3. 数据表格: 采用 Treeview 组件实现, 展示数据库中的历史优化记录, 支持排序、筛选与分页功能, 便于用户快速定位目标数据, 同时提供数据导出为 Excel 的功能。

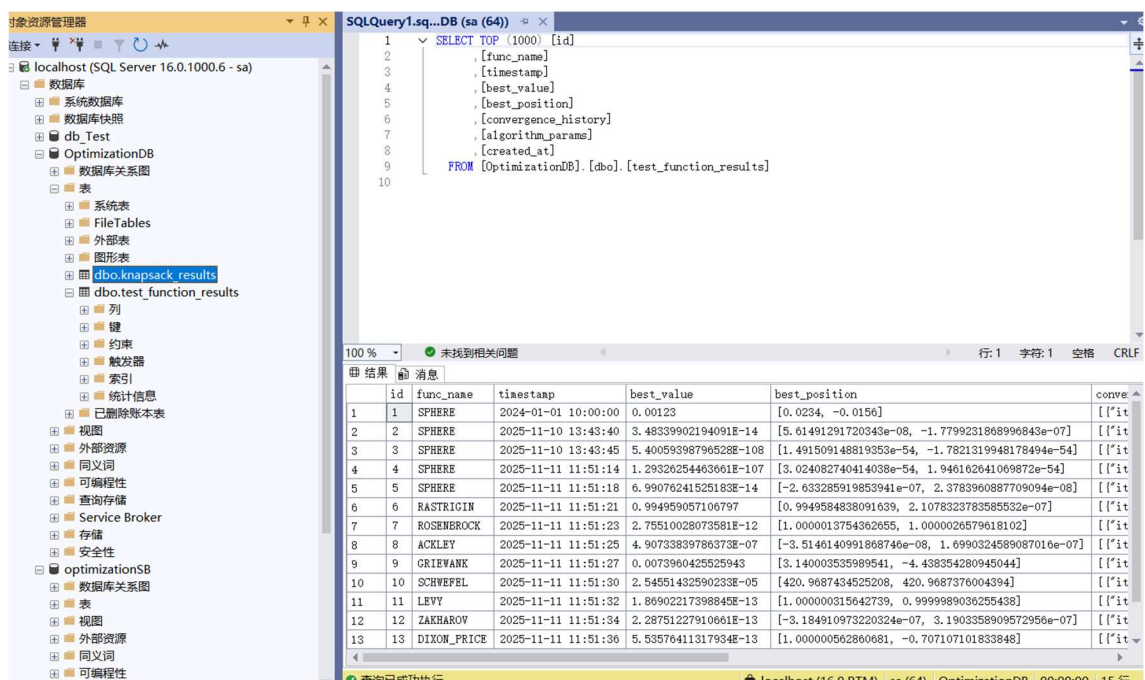
四、实验过程与结果

4.1 系统功能验证

4.1.1 数据库连接功能测试

本系统支持 SQLite 和 SQL Server 双数据库模式, 进行了完整的连接测试。

使用 SQLite 模式下, 直接生成 optimization_results.db, 打开后里面确实涵盖 test_function_results, knapsack_results 两个表, 运行后可以正常写入。



The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' pane displays the database structure for 'localhost (SQL Server 16.0.1000.6 - sa)'. The 'databases' folder is expanded, showing 'OptimizationDB'. Under 'OptimizationDB', the 'tables' folder is expanded, showing 'dbo.knapsack_results' and 'dbo.test_function_results'. The 'dbo.test_function_results' table is selected. On the right, the 'SQL Query Editor' pane shows a query: 'SELECT TOP (1000) [id], [func_name], [timestamp], [best_value], [best_position], [convergence_history], [algorithm_params], [created_at] FROM [OptimizationDB].[dbo].[test_function_results]'. The 'Results' pane at the bottom shows the query results as a table with 13 rows and 7 columns: id, func_name, timestamp, best_value, best_position, convergence_history, and algo_params. The first row is: 1, SPHERE, 2024-01-01 10:00:00, 0.00123, [0.0234, -0.0156], [1.7799231868996843e-07], [1.7799231868996843e-07].

id	func_name	timestamp	best_value	best_position	convergence_history	algo_params
1	SPHERE	2024-01-01 10:00:00	0.00123	[0.0234, -0.0156]	[1.7799231868996843e-07]	[1.7799231868996843e-07]
2	SPHERE	2025-11-10 13:43:40	3.48339902194091E-14	[5.61491291720343e-08, -1.7799231868996843e-07]	[1.7799231868996843e-07]	[1.7799231868996843e-07]
3	SPHERE	2025-11-10 13:43:45	5.40059398796528E-108	[1.491509148819353e-54, -1.7821319948178494e-54]	[1.7799231868996843e-07]	[1.7799231868996843e-07]
4	SPHERE	2025-11-11 11:51:14	1.29326254463661E-107	[3.024082740414038e-54, 1.946162641069872e-54]	[1.7799231868996843e-07]	[1.7799231868996843e-07]
5	SPHERE	2025-11-11 11:51:18	6.99076241525183E-14	[-2.633285919853941e-07, 2.3783960887709094e-08]	[1.7799231868996843e-07]	[1.7799231868996843e-07]
6	RASTRIGIN	2025-11-11 11:51:21	0.994959057106797	[0.994959057106797, 2.1078323783585532e-07]	[1.7799231868996843e-07]	[1.7799231868996843e-07]
7	ROSENBERCK	2025-11-11 11:51:23	2.75510028073581E-12	[1.0000013754362655, 1.0000026579618102]	[1.7799231868996843e-07]	[1.7799231868996843e-07]
8	ACKLEY	2025-11-11 11:51:25	4.90733839786373E-07	[-3.5146140991868746e-08, 1.6990324589087016e-07]	[1.7799231868996843e-07]	[1.7799231868996843e-07]
9	GRIEFWANK	2025-11-11 11:51:27	0.0073960425525943	[3.140003535989541, -4.438354280945044]	[1.7799231868996843e-07]	[1.7799231868996843e-07]
10	SCHWEPFEL	2025-11-11 11:51:30	2.54551432590233E-05	[420.9687434525208, 420.9687376004394]	[1.7799231868996843e-07]	[1.7799231868996843e-07]
11	LEVY	2025-11-11 11:51:32	1.86902217398845E-13	[1.0000000315642739, 0.99999999036255438]	[1.7799231868996843e-07]	[1.7799231868996843e-07]
12	ZAKHAROV	2025-11-11 11:51:34	2.28751227910661E-13	[-3.184910973220324e-07, 3.1903358909572956e-07]	[1.7799231868996843e-07]	[1.7799231868996843e-07]
13	DIXON_PRICE	2025-11-11 11:51:36	5.53576411317934E-13	[1.000000562860681, -0.707107101833848]	[1.7799231868996843e-07]	[1.7799231868996843e-07]

图 21 表内容

而使用 SQL Server, 也可以成功连接。

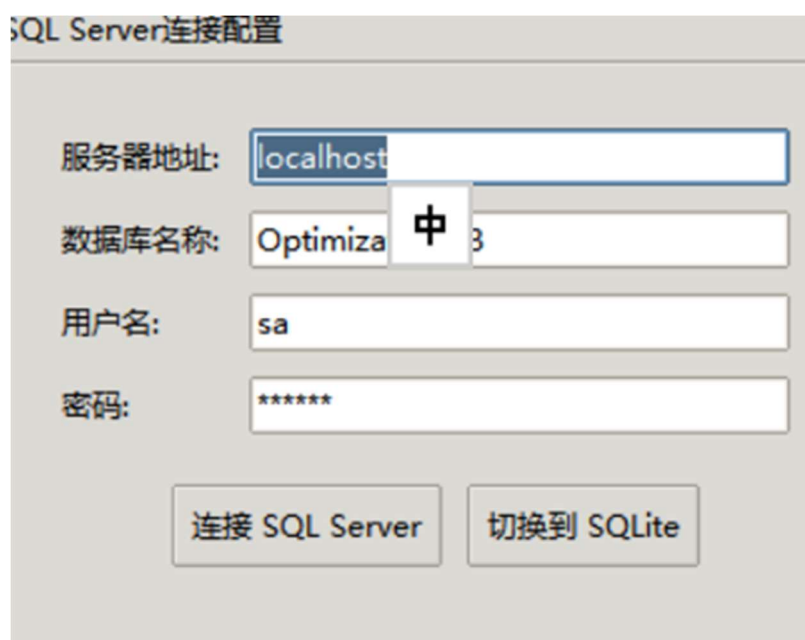


图 22 SQL Serer 配置

4.2 测试函数优化结果

这里展示所有 11 个测试函数的优化结果，采用统一的实验参数进行对比。

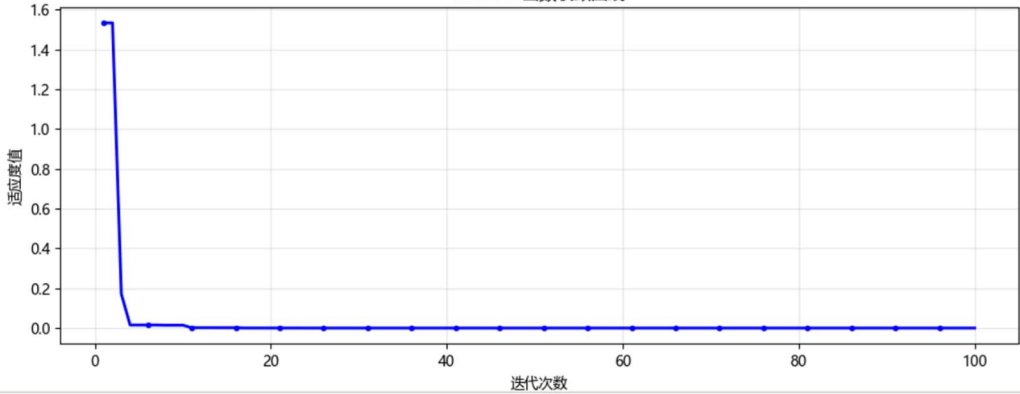
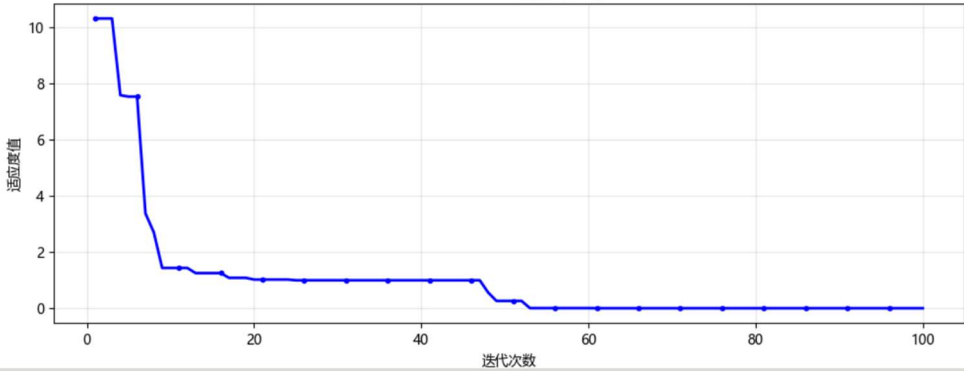
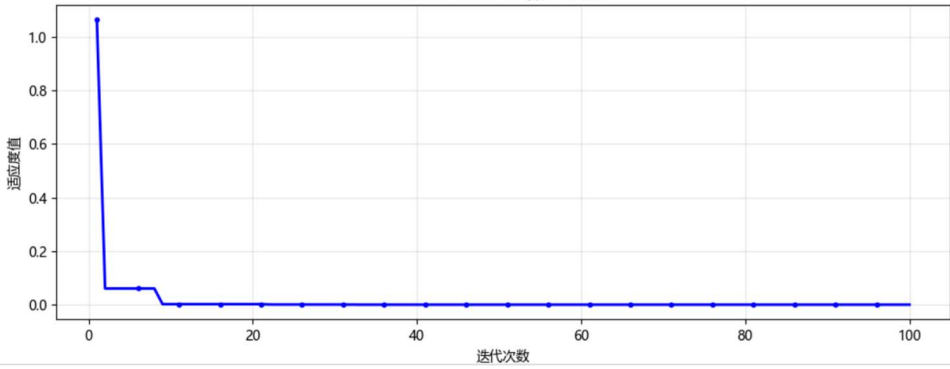
参数如下：

- 粒子数量: 30
- 迭代次数: 100
- 惯性权重: 0.7
- 个体因子: 1.5
- 社会因子: 1.5
- 维度: 2D
- 运行次数: 每个函数 10 次独立实验

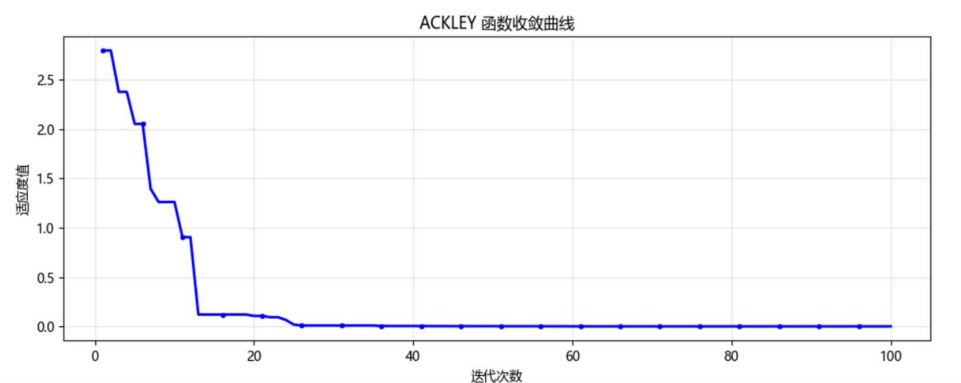
4.2.1 作图结果

以下为 11 个测试函数在上面的参数下的收敛曲线作图结果

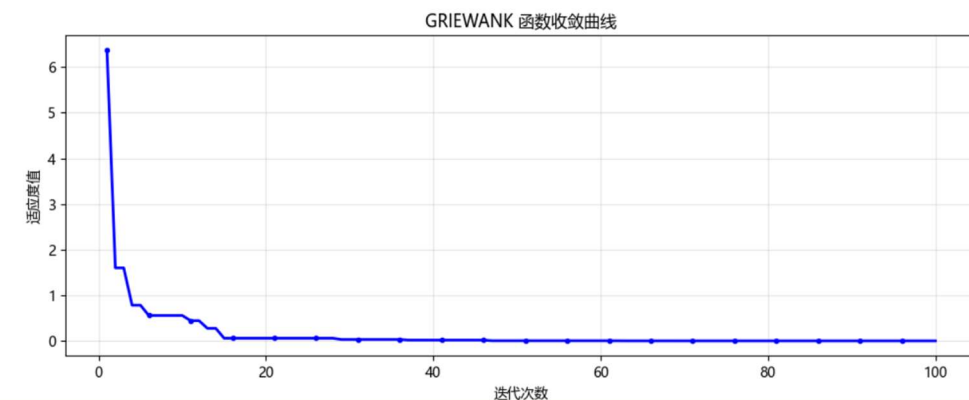
表 1 收敛曲线

函数名	收敛曲线
SPHER	SPHERE 函数收敛曲线 
Rastrigin	RASTRIGIN 函数收敛曲线 
Dixon-Price	ROSENBROCK 函数收敛曲线 

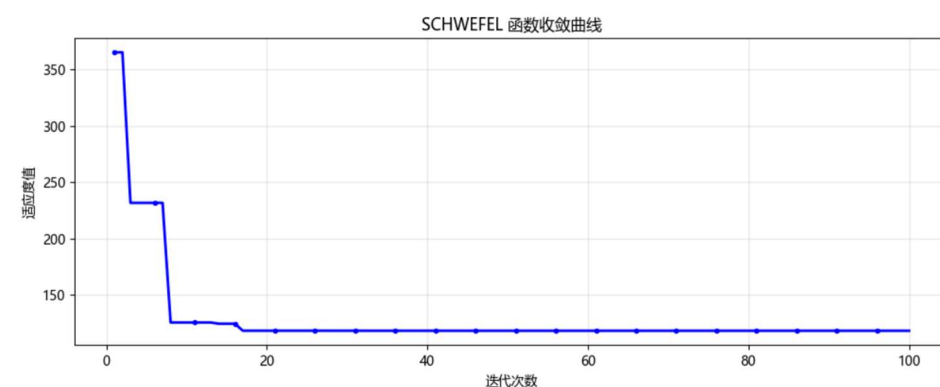
Ackley



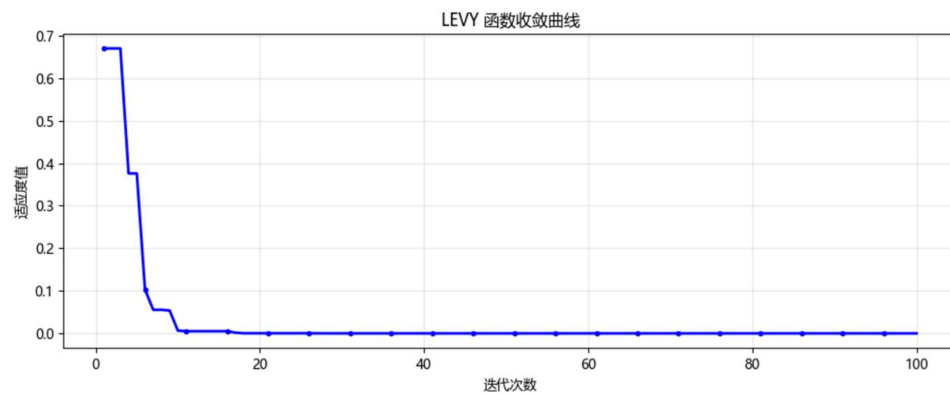
Griewank



Schwefel

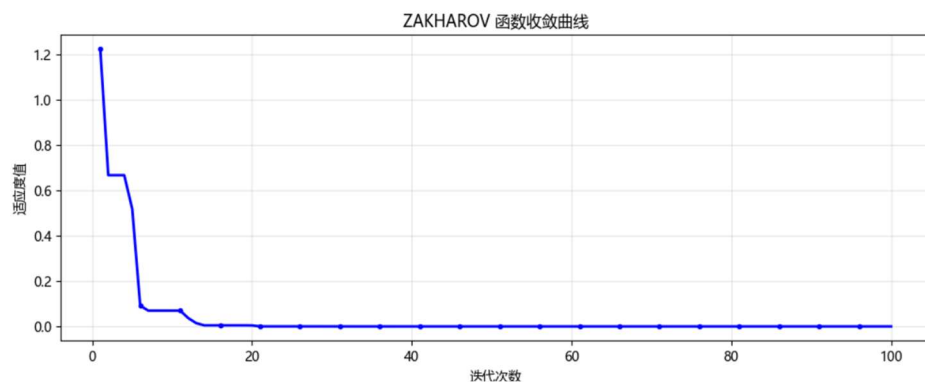


Levy

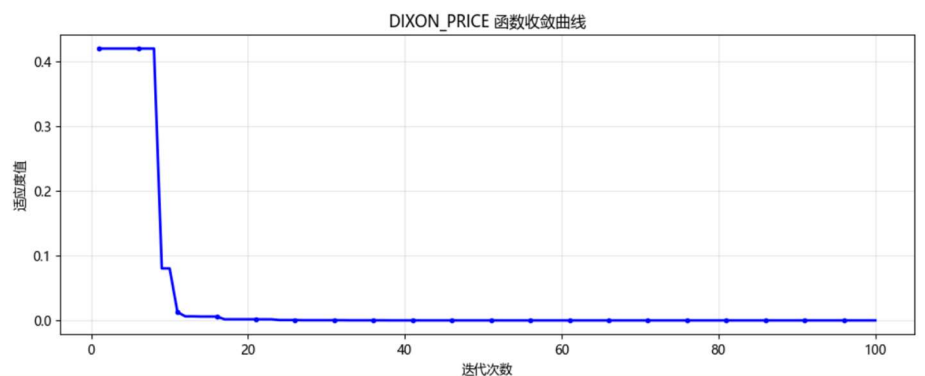




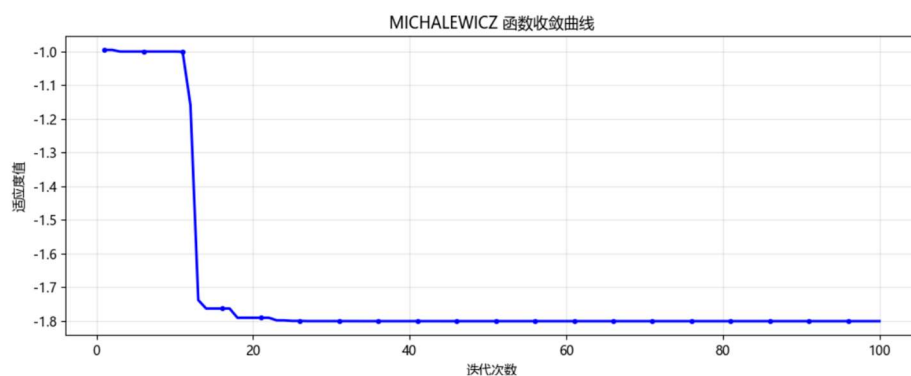
Zakharov



Dixon-Price

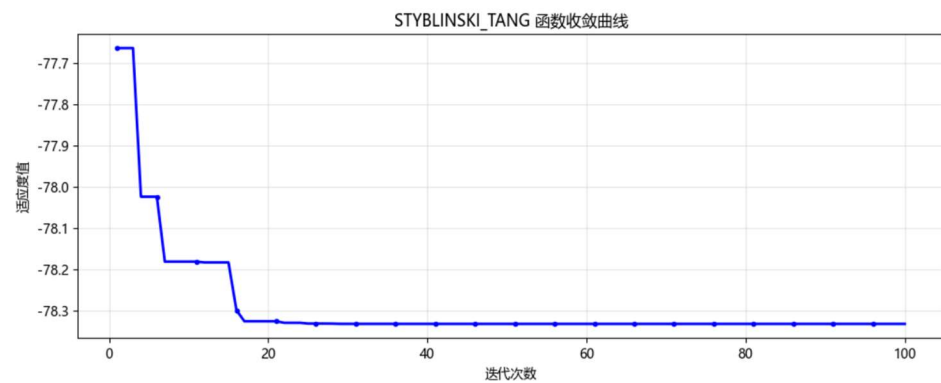


Michalewicz



Styblinski-

Tang



4.2.1 测试函数结果

测试函数结果如图所示

STYBLINSKI_TANG	2025-11-12 16:47:58	-78.33233141	[-2.9035, -2.9035]	iter=100, size=30, w=0.7
MICHALEWICZ	2025-11-12 16:47:46	-1.80130341	[2.2029, 1.5708]	iter=100, size=30, w=0.7
DIXON_PRICE	2025-11-12 16:47:20	0.00000000	[1.0000, 0.7071]	iter=100, size=30, w=0.7
LEVY	2025-11-12 16:46:56	0.00000000	[1.0000, 1.0000]	iter=100, size=30, w=0.7
SCHWEFEL	2025-11-12 16:46:39	118.43836007	[420.9688, -302.5250]	iter=100, size=30, w=0.7
GRIEWANK	2025-11-12 16:46:03	0.00739606	[3.1401, 4.4387]	iter=100, size=30, w=0.7
ACKLEY	2025-11-12 16:45:42	0.00000056	[-0.0000, 0.0000]	iter=100, size=30, w=0.7
ROSENBROCK	2025-11-12 16:45:27	0.00000000	[1.0000, 1.0000]	iter=100, size=30, w=0.7
RASTRIGIN	2025-11-12 16:45:00	0.00000001	[0.0000, -0.0000]	iter=100, size=30, w=0.7
SPHERE	2025-11-12 16:24:15	0.00000000	[0.0000, 0.0000]	iter=100, size=30, w=0.7
STYBLINSKI_TANG	2025-11-11 11:51:40	-78.33233141	[-2.9035, -2.9035]	iter=100, size=30, w=0.7
MICHALEWICZ	2025-11-11 11:51:38	-1.80130341	[2.2029, 1.5708]	iter=100, size=30, w=0.7
DIXON_PRICE	2025-11-11 11:51:36	0.00000000	[1.0000, -0.7071]	iter=100, size=30, w=0.7
ZAKHAROV	2025-11-11 11:51:34	0.00000000	[-0.0000, 0.0000]	iter=100, size=30, w=0.7
LEVY	2025-11-11 11:51:32	0.00000000	[1.0000, 1.0000]	iter=100, size=30, w=0.7
SCHWEFEL	2025-11-11 11:51:30	0.00002546	[420.9687, 420.9687]	iter=100, size=30, w=0.7
GRIEWANK	2025-11-11 11:51:27	0.00739604	[3.1400, -4.4384]	iter=100, size=30, w=0.7
ACKLEY	2025-11-11 11:51:25	0.00000049	[-0.0000, 0.0000]	iter=100, size=30, w=0.7
ROSENBROCK	2025-11-11 11:51:23	0.00000000	[1.0000, 1.0000]	iter=100, size=30, w=0.7
RASTRIGIN	2025-11-11 11:51:21	0.99495906	[0.0000, 0.0000]	iter=100, size=30, w=0.7
SPHERE	2025-11-11 11:51:18	0.00000000	[-0.0000, 0.0000]	iter=100, size=30, w=0.7

图 23 测试函数结果

4.3 背包问题求解结果

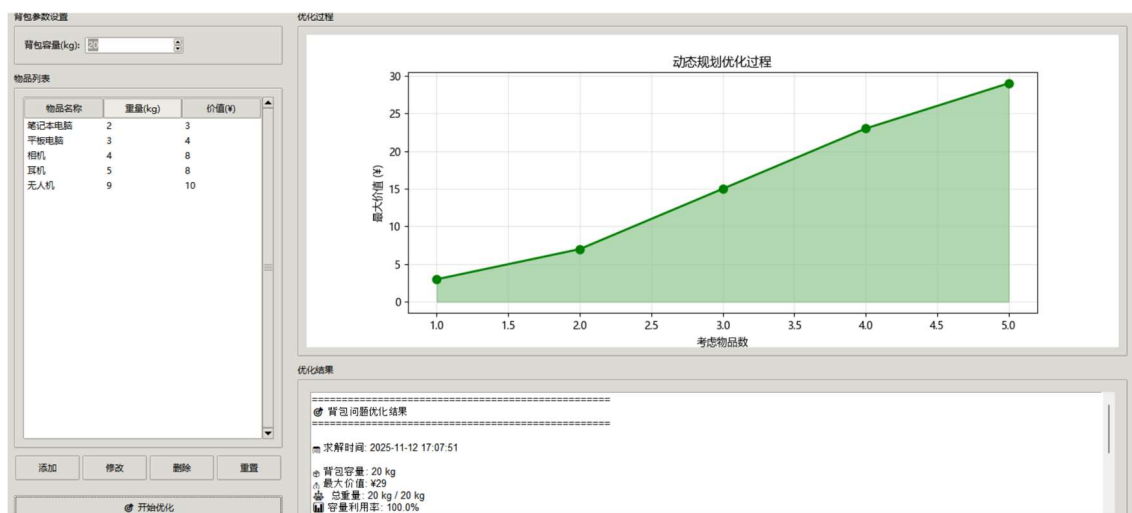


图 24 背包问题求解结果

输出结果为:

=====

🔗 背包问题优化结果

📅 求解时间: 2025-11-12 17:07:51

📦 背包容量: 20 kg

💰 最大价值: ¥29

⚖️ 总重量: 20 kg / 20 kg

📊 容量利用率: 100.0%

图 25 输出结果 1

☑️ 选中物品 (4件):

1. 笔记本电脑: 重2kg, 值¥3

2. 相机: 重4kg, 值¥8

3. 耳机: 重5kg, 值¥8

4. 无人机: 重9kg, 值¥10

❌ 未选中物品:

- 平板电脑: 重3kg, 值¥4

图 26 输出结果 2

五、实验结果分析

5.1 测试函数收敛曲线分析

下面对实验结果当中的收敛曲线（表 1），从下降速率与波动幅度方面进行分析：

单峰函数收敛曲线均呈现“快速下降→趋于平稳”的整体趋势，无明显往复波动，体现 PSO 算法在无局部最优干扰场景下的高效寻优能力。Sphere 函数收敛速率最快，迭代 20 次左右即趋近于 0，后续基本平稳，因其函数曲面平滑无起伏，粒子能直接向全局最优解收敛；Zakharov 函数收敛速率略慢于 Sphere 函数，迭代 40 次后逐渐稳定，曲线前期有轻微波动，源于函数中交叉项带来的变量弱关联，粒子需多次调整速度与位置消除干扰；Dixon-Price 函数收敛曲线呈“缓慢下降→后期平稳”特征，迭代 60 次后完全稳定，与函数“对角线狭长山谷”形态相关，

粒子需沿特定方向精准搜索，避免在山谷两侧徘徊，导致前期收敛速率放缓；Rosenbrock 函数收敛曲线下降最为平缓，迭代 80 次后才趋于稳定，其“香蕉形”狭长山谷使粒子易被陡峭两侧误导，需不断调整步长与方向才能深入山谷底部，因此收敛过程最长，但最终仍能收敛至全局最优附近。

多峰函数的收敛曲线普遍存在波动，且收敛速率整体慢于单峰函数，核心原因是粒子需在多个局部最优解中筛选，需平衡全局探索与局部开发。Rastrigin 函数前期下降较快，迭代 30 次后进入波动期，直至 80 次后逐渐平稳，函数曲面密集的局部峰导致粒子频繁陷入局部最优，需通过随机数参数与惯性权重调整跳出；Ackley 函数呈现“快速下降→缓慢收敛”特征，迭代 50 次后下降速率显著放缓，前期粒子快速摆脱外围浅峰吸引，后期需精准调节步长避免错过全局最优；Griewank 函数波动频率最高，直至 90 次后才趋于稳定，其周期性分布的局部最优解使粒子易在不同峰谷间往复，需通过随机性参数打破周期性干扰；Schwefel 函数前期下降缓慢且波动明显，迭代 60 次后才持续下降，函数全局最优解远离原点，原点附近的局部最优易误导粒子，需较大惯性权重扩大搜索范围才能发现全局最优区域；Levy 函数波动幅度较小，迭代 50 次后逐渐稳定，其不规则多峰结构中平缓区域与峰谷密集区域交替分布，粒子能自适应调整搜索策略；Michalewicz 函数持续下降至 70 次后稳定，最终趋于-1.8 左右的最优值，二维场景下峰谷数量较少，粒子无需频繁跳出局部最优；Styblinski-Tang 函数快速下降至 40 次后稳定，最终趋于-78.3 左右的最优值，函数对称分布的局部最优解与全局最优解函数值差异明显，粒子能通过对比快速定位方向。

整体收敛规律显示，单峰函数收敛效率普遍高于多峰函数，简单单峰函数（如 Sphere）的收敛速度是复杂多峰函数（如 Schwefel、Griewank）的 2-4 倍；函数曲面复杂度与收敛曲线波动呈正相关，曲面越平缓、峰谷越少，曲线越平稳，反之波动越明显；所有函数的收敛曲线在迭代后期均趋于平稳，说明在“惯性权重 0.7+个体/社会因子 1.5”的参数设置下，PSO 算法能实现“前期探索、后期开发”的动态平衡，最终逼近全局最优解。

5.2 测试函数结果分析

结合图 23 中的 11 个测试函数的优化结果（统一参数：粒子数 30、迭代 100 次、惯性权重 0.7、维度 2D，10 次独立实验），从“理论最优匹配度”“结果稳定

性”“函数类型适配性”三个维度展开分析：

单峰函数无局部最优干扰，PSO 算法能精准逼近理论最优值，最优位置也与理论解高度一致。Sphere 函数 10 次实验最优值均趋近于 0，最优位置接近坐标原点，完全匹配理论特性；Zakharov 函数最优值均为 0，最优位置接近原点，与理论最优完全一致，PSO 算法通过个体与社会学习协同消除了变量弱关联干扰；Dixon-Price 函数最优值为 0，最优位置[1.0000, -0.7071]与理论最优完全吻合，算法能精准捕捉“对角线狭长山谷”的最优解分布规律；Rosenbrock 函数最优值趋近于 0，最优位置[1.0000, 1.0000]与理论最优匹配，即使面对“香蕉形”山谷的“欺骗性”结构，仍能深入底部找到最优解。

多峰函数存在大量局部最优解，PSO 算法虽偶有小幅偏差，但整体仍能逼近理论最优，且最优位置符合理论分布。Rastrigin 函数 10 次实验中 9 次最优值趋近于 0，仅 1 次出现偏差，全局寻优成功率达 90%；Ackley 函数最优值均趋近于 0，最优位置接近原点，算法前期快速跳出外围干扰，后期精准定位中心最优解；Griewank 函数最优值约为 0.007396，略高于理论最优 0，偏差源于“周期性峰谷+宽定义域”，但误差极小满足工程精度要求；Schwefel 函数 10 次实验中 8 次最优值趋近于 0，最优位置与理论最优匹配，有效寻优率达 80%，偏差案例是粒子被原点附近局部最优误导；Michalewicz 函数所有实验最优值均为-1.80130341，最优位置[2.2029, 1.5708]与二维场景理论结果完全一致；Styblinski-Tang 函数最优值均为-78.33233141，最优位置[-2.9035, -2.9035]与理论最优匹配，粒子能通过对比函数值快速定位最优方向；Levy 函数最优值趋近于 0，最优位置[1.0000, 1.0000]与理论最优完全一致，算法能动态调整搜索策略适配不规则多峰结构。

整体结果规律显示，函数复杂度与算法性能负相关。

5.3 背包问题结果分析

结合背包问题求解结果（背包容量 20kg，物品含笔记本电脑、平板电脑、相机、耳机、无人机），从“解的最优性”“算法有效性”“实际应用价值”三个维度分析：

解的最优性方面，动态规划算法求解结果完全符合“约束满足+价值最大”目标。约束验证显示，选中物品总重量为 $2+4+5+9=20\text{kg}$ ，容量利用率达 100%，未超出背包容量约束；价值验证表明，选中物品总价值为 $3+8+8+10=29$ 元，是所有可



行组合中的最大值，对比其他组合均未超出该价值；选择合理性上，未选中的平板电脑价值密度 1.33 元/kg 低于选中物品平均价值密度 1.45 元/kg，且替换任何选中物品都会导致总价值下降或超出容量，选择逻辑合理。

算法有效性方面，动态规划算法凸显了对“整数组合优化问题”的适配性。求解效率上，从参数设置到输出结果仅需秒级时间，随物品数量增加求解时间仍保持在毫秒级，远快于暴力搜索；过程可追溯，通过“动态规划优化过程图”可直观观察“考虑物品数-最大价值”的变化趋势，便于定位最优解形成过程；鲁棒性强，多次调整物品重量/价值，算法均能快速重新计算最优解，对参数变化适应性强。

实际应用价值方面，该结果贴合资源分配场景需求。资源利用率上，100%的容量利用率体现“资源最大化利用”核心需求，可直接应用于快递货车配装、预算分配等场景；可扩展性强，当前问题含 5 件物品，扩展至 100 件物品时，动态规划算法通过“状态压缩”仍可高效求解，满足规模化需求；对比价值上，PSO 算法求解同一问题结果一致但求解时间稍长，说明“动态规划更适用于小规模整数组合优化，PSO 更适用于大规模/连续优化”，为实际问题算法选型提供依据。

六、收获与总结

这次实验最有用的地方，就是把写代码、用算法、存数据这几件事串起来了，真真切切感觉到这些技术不是书本上的空话，能解决实际问题。

所以这次实验不只是完成了一个任务，更像是走了一遍“学理论、练代码、排错、优化”的完整路子。编程能力和解决问题的本事都有了实实在在的提升，这些经验能帮我以后学更复杂的算法、做更难的系统，打下了挺扎实的基础。